CENTRO UNIVERSITÁRIO DINÂMICA DAS CATARATAS
CURSO DE SISTEMAS DE INFORMAÇÃO

ILAN WENDLING THOELE

COMUNICAÇÃO SÍNCRONA VIA REST E ASSÍNCRONA ORIENTADA A EVENTOS: UMA ANÁLISE EXPERIMENTAL NO PROCESSAMENTO DE REGISTROS

FOZ DO IGUAÇU
2026

ILAN WENDLING THOELE

COMUNICAÇÃO SÍNCRONA VIA REST E ASSÍNCRONA ORIENTADA A EVENTOS: UMA ANÁLISE EXPERIMENTAL NO PROCESSAMENTO DE REGISTROS

Trabalho de Conclusão de Curso apresentado como requisito obrigatório para obtenção do título de Bacharel em Sistemas de Informação do Centro Universitário Dinâmica das Cataratas.

Orientador: Professora Alessandra Bussador

FOZ DO IGUAÇU

2026

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA

Para dar transparência ao processo de elaboração deste Trabalho de Conclusão de Curso, declara-se o uso de ferramentas de Inteligência Artificial Generativa (IAG), com as finalidades discriminadas a seguir.

Ferramentas utilizadas: Claude (Anthropic) e Codex (OpenAI), modelos de linguagem de propósito geral.

Finalidade do uso: apoio à reorganização do tema, estruturação textual, revisão gramatical e estilística, levantamento e conferência de fontes, formatação acadêmica, elaboração e revisão de diagramas, discussão do protocolo experimental, geração e revisão de código, preparação e execução assistida de testes e da campanha, organização e agregação dos dados, redação assistida dos resultados e conclusões, e verificação da consistência entre documentação e implementação.

Os resultados de execução apresentados devem ser rastreáveis aos dados brutos e aos comandos registrados. Testes de software e ensaios exploratórios não substituem a coleta experimental definitiva. O uso dessas ferramentas não autoriza fabricar resultados, alterar medições ou citar trabalhos inexistentes. A conferência das fontes, a aprovação do método e a interpretação científica permanecem sob responsabilidade do autor.

O autor assume responsabilidade pelo conteúdo submetido, pelas decisões metodológicas, pelo código, pelos dados e pelas conclusões, inclusive pelos trechos elaborados com auxílio de IAG, cuja revisão crítica deve preceder a entrega à instituição.

RESUMO

Este trabalho compara experimentalmente duas implementações Java de processamento de registros de auditoria: uma API REST síncrona e uma solução assíncrona com Apache Kafka e consumidor. As variantes compartilham contrato, validação, hash e persistência em PostgreSQL. Após calibração e nove validações do piloto R2, uma campanha local com k6 observou 65 UUIDs em treze condições e cinco tentativas por condição, incluindo cargas constantes, rajada e indisponibilidades. Houve 64 coletas completas e uma parcial qualificada por emenda analítica, preservando o veredito original e sem imputar um instante de commit ausente. A análise descritiva por execução e par separa resposta HTTP, conclusão, vazão, recursos e destinos. A assíncrona apresentou menor p95 de aceite e maior uso de CPU/memória nos pares comparáveis, mas maior p95 de conclusão nas condições sem falha. Com consumidor parado, todos os aceites foram persistidos ao fim; com banco parado, 14 eventos aceitos terminaram em fila de falhas terminais e somente uma de cinco execuções assíncronas satisfez o critério de recuperação sustentada na janela ativa. Os resultados sustentam compromissos contextuais, não superioridade geral. Um computador compartilhado, broker único, dados sintéticos, cinco tentativas e a emenda pós-hoc limitam a generalização.

Palavras-chaves: Arquitetura orientada a eventos. REST. Registros de auditoria. Apache Kafka.

LISTA DE FIGURAS

FIGURA 1 – Visão geral dos caminhos síncrono e assíncrono em Java 30
FIGURA 2 – Casos de uso da aplicação operacional 37
FIGURA 3 – Casos de uso da camada de avaliação 37
FIGURA 4 – Atividade síncrona: transação, duplicidade e conflito 41
FIGURA 5 – Atividade da API produtora e confirmação de publicação 42
FIGURA 6 – Atividade do consumidor, retry e confirmação da origem 43
FIGURA 7 – Classes de contrato e persistência 44
FIGURA 8 – Classes de aplicação e processamento Java 45
FIGURA 9 – Sequência síncrona com inserção idempotente 46
FIGURA 10 – Sequência assíncrona com ACK e tratamento de falhas 47
FIGURA 11 – Dependências entre os cinco módulos Maven 48
FIGURA 12 – Arquitetura da variante síncrona 48
FIGURA 13 – Arquitetura da variante assíncrona e DLQ 49
FIGURA 14 – Implantação real do protótipo no Docker Compose 49
FIGURA 15 – Modelo conceitual do núcleo existente 51
FIGURA 16 – Modelo conceitual da extensão de observabilidade proposta 52
FIGURA 17 – Modelo lógico do banco implementado 52
FIGURA 18 – Modelo lógico da extensão proposta, separado do esquema atual . . 54
FIGURA 19 – WF01: wireframe proposto para envio de registros 58
FIGURA 20 – WF02: wireframe proposto para métricas 59
FIGURA 21 – WF03: wireframe proposto para backlog e recuperação 59
FIGURA 22 – WF04: wireframe proposto para comparação experimental 60

LISTA DE TABELAS

TABELA 1 – Requisitos funcionais e critérios de aceitação 31
TABELA 2 – Requisitos não funcionais 34
TABELA 3 – Rastreabilidade funcional e comportamental 50
TABELA 4 – Dicionário do registro persistido 54
TABELA 5 – Dicionário proposto dos logs de processamento 55
TABELA 6 – Dicionário proposto das tentativas de recuperação 56
TABELA 7 – Dicionário proposto das amostras de métricas 57
TABELA 8 – Módulos Java e responsabilidades efetivas 61
TABELA 9 – Latência: média e desvio-padrão dos p95 de cada execução, em ms . 69
TABELA 10 – Diferenças pareadas do p95 HTTP: assíncrona menos síncrona, em ms 69
TABELA 11 – Vazão concluída na janela e erros HTTP por execução 70
TABELA 12 – Destinos ao fim da observação: totais das cinco tentativas por condição 70
TABELA 13 – Recursos do caminho de processamento: média e desvio-padrão das médias por execução 71
TABELA 14 – Recuperação observada após o comando de retomada, em segundos . 72

SUMÁRIO

1 INTRODUÇÃO 9
1.1 Problema de pesquisa 10
1.2 Justificativa 10
1.3 Objetivos 11
1.3.1 Objetivo geral 11
1.3.2 Objetivos específicos 11
1.4 Hipóteses de trabalho 11
1.5 Delimitação 11
1.6 Organização do trabalho 12

2 REVISÃO BIBLIOGRÁFICA 13
2.1 Registros e trilhas de auditoria 13
2.2 Comunicação em sistemas distribuídos 14
2.2.1 HTTP, REST e comunicação síncrona 14
2.2.2 Mensageria e comunicação assíncrona 15
2.3 Arquitetura orientada a eventos 16
2.4 Apache Kafka 17
2.5 Desempenho, resiliência e observabilidade 18
2.6 Trabalhos relacionados 19
2.7 Síntese 20

3 METODOLOGIA 21
3.1 Tipo de pesquisa 21
3.1.1 Natureza da pesquisa 21
3.1.2 Abordagem metodológica 21
3.1.3 Procedimentos técnicos 21
3.2 Procedimentos metodológicos 22
3.2.1 Métodos de coleta de informações 22
3.2.2 Métodos de análise das informações 23
3.3 Metodologia de desenvolvimento do sistema 23
3.3.1 Modelo de desenvolvimento de software 23
3.3.2 Tecnologias utilizadas 23
3.3.3 Arquitetura do sistema 24
3.4 Procedimentos de teste e validação 24
3.4.1 Tipos de testes aplicados 24

3.4.2 Cenários de teste . . . 25
3.4.3 Métricas coletadas . . . 26
3.4.4 Emenda analítica após início da campanha . . . 27
3.4.5 Critérios de aceitação . . . 28
3.4.6 Protocolo de execução e reprodutibilidade . . . 28
3.4.7 Ameaças à validade . . . 29

4 DESENVOLVIMENTO DO SISTEMA . . . 30
4.1 Visão geral do sistema . . . 30
4.2 Levantamento e especificação de requisitos . . . 31
4.2.1 Requisitos funcionais . . . 31
4.2.2 Requisitos não funcionais . . . 33
4.2.3 Regras de negócio . . . 35
4.3 Modelagem UML . . . 36
4.3.1 Diagrama de casos de uso . . . 36
4.3.1.1 UC01 — Registrar auditoria sincronamente . . . 36
4.3.1.2 UC02 — Aceitar auditoria assincronamente . . . 38
4.3.1.3 UC03 — Consumir e persistir evento . . . 39
4.3.1.4 UC04 — Consultar prontidão . . . 39
4.3.1.5 UC05 — Executar cenário de avaliação (planejado) . . . 39
4.3.1.6 UC06 — Consolidar e consultar indicadores (planejado) . . . 40
4.3.1.7 UC07 — Consultar registro por UUID . . . 40
4.3.2 Diagramas de atividades . . . 40
4.3.3 Diagramas de classes . . . 44
4.3.4 Diagramas complementares . . . 45
4.3.4.1 Diagrama de implantação . . . 45
4.3.5 Rastreabilidade entre requisitos, regras e UML . . . 50
4.4 Modelagem de banco de dados . . . 50
4.4.1 Modelo conceitual (MER) . . . 51
4.4.2 Modelo lógico . . . 51
4.4.3 Modelo físico implementado . . . 53
4.4.4 Extensão lógica opcional . . . 53
4.4.5 Dicionário de dados . . . 54
4.4.5.1 Tabela audit_records — existente . . . 54
4.4.5.2 Tabela processing_log — proposta . . . 55
4.4.5.3 Tabela retry_log — proposta . . . 56
4.4.5.4 Tabela metrics — proposta . . . 56
4.5 Prototipação da interface . . . 57
4.5.1 Envio de registro . . . 57
4.5.2 Visualização de métricas . . . 58

4.5.3 Acompanhamento de backlog . . . 58
4.5.4 Comparação entre variantes . . . 60
4.6 Implementação do sistema . . . 60
4.6.1 Tecnologias e módulos . . . 60
4.6.2 Comportamento implementado . . . 61
4.6.3 Verificação funcional e limites da evidência . . . 62
4.6.4 Rastreabilidade da entrega . . . 62

5 ANÁLISE DOS RESULTADOS . . . 64
5.1 Ferramentas de carga e piloto . . . 64
5.1.1 Validação inicial dos instrumentos . . . 65
5.1.2 Calibração e validação da revisão reduzida . . . 65
5.2 Modelo comum de registros: objetivo a . . . 66
5.3 Variante síncrona: objetivo b . . . 67
5.4 Variante assíncrona: objetivo c . . . 67
5.5 Execução dos cenários: objetivo d . . . 67
5.6 Métricas comparativas: objetivo e . . . 69
5.6.1 Resposta HTTP e conclusão da persistência . . . 69
5.6.2 Vazão, erros e destinos . . . 70
5.6.3 CPU e memória . . . 71
5.6.4 Recuperação e persistência após falha . . . 72
5.6.5 Sensibilidade e integridade temporal . . . 73
5.7 Vantagens, desvantagens e hipóteses: objetivo f . . . 73
5.8 Limitações . . . 74

6 CONCLUSÃO . . . 75
6.1 Objetivos e hipóteses . . . 75
6.2 Limitações e recomendações . . . 76
6.3 Trabalhos futuros . . . 77

REFERÊNCIAS . . . 78

1 INTRODUÇÃO

Sistemas de informação registram continuamente ações executadas por usuários, serviços e componentes de infraestrutura. Autenticações, alterações cadastrais, concessões de permissão e mudanças de configuração são exemplos de ocorrências que podem compor uma trilha de auditoria. Amir-Mohammadian, Chong e Skalka (2016) tratam a auditoria como mecanismo de segurança retrospectiva e mostram que a utilidade do registro depende de sua relação correta com a execução observada. Assim, produção consistente, integridade e recuperação posterior são propriedades relevantes para o uso desses dados como evidência.

Em aplicações distribuídas, o componente que produz uma ocorrência nem sempre é o mesmo que valida, enriquece e persiste o registro de auditoria. Consequentemente, é necessário escolher uma forma de comunicação entre esses participantes. Em uma interação síncrona, o produtor envia uma requisição e aguarda a conclusão do processamento. APIs REST utilizam recursos e interfaces uniformes para organizar interações sobre a Web (FIELDING; TAYLOR, 2002; PAUTASSO; ZIMMERMANN; LEYMANN, 2008). Essa alternativa oferece confirmação imediata, porém cria dependência temporal entre os participantes.

Na comunicação assíncrona, a aceitação e o processamento podem ocorrer em momentos distintos. Um produtor publica uma mensagem, um broker a retém e um consumidor realiza o processamento. Esse mecanismo reduz o acoplamento temporal e permite absorver rajadas de carga, mas introduz questões de reentrega, duplicidade, ordenação, observabilidade e consistência temporal (KARABEY AKSAKALLI et al., 2021; HELLAND, 2007). Em uma arquitetura orientada a eventos, os componentes produzem e reagem a representações de fatos ocorridos. A arquitetura não deve ser confundida com uma tecnologia específica: o Apache Kafka é uma implementação de mensageria distribuída baseada em log que pode integrar uma solução orientada a eventos (KREPS; NARKHEDE; RAO, 2011).

Estudos empíricos indicam que os efeitos dessas decisões dependem do contexto. Cabane e Farias (2024) compararam uma aplicação monolítica e uma implementação orientada a eventos e observaram diferenças em tempo de resposta e consumo de recursos. Kazanavičius e Mažeika (2023) avaliaram REST, RabbitMQ, Kafka, gRPC e GraphQL e concluíram que a seleção deve considerar necessidades e critérios específicos. Esses trabalhos desaconselham a suposição de que uma alternativa seja universalmente superior.

Este estudo utiliza o processamento de registros de auditoria como cenário controlado. A funcionalidade é implementada em duas variantes com contrato, validação, cálculo de hash e persistência equivalentes. Na primeira, o cliente aguarda o processamento completo por uma API REST. Na segunda, a API publica um evento no Kafka

e um consumidor realiza a persistência. Essa equivalência funcional permite comparar as duas configurações sem introduzir regras de negócio distintas. A topologia assíncrona acrescenta componentes e consumo de recursos; esses custos fazem parte da comparação e impedem atribuir todas as diferenças exclusivamente ao estilo de comunicação. REST não implica, por definição, conclusão síncrona: essa condição é uma decisão da variante construída neste trabalho.

1.1 PROBLEMA DE PESQUISA

A adoção de comunicação assíncrona pode reduzir o tempo pelo qual o cliente permanece bloqueado e permitir retenção de mensagens durante indisponibilidades do consumidor. Em contrapartida, adiciona broker, consumidor, gerenciamento de offsets e necessidade de idempotência. A comunicação síncrona apresenta um fluxo mais direto, mas o produtor depende da disponibilidade e do tempo de processamento do serviço de auditoria.

Entre os trabalhos analisados, foram identificadas avaliações de arquiteturas monolíticas e distribuídas, de diferentes tecnologias de comunicação e do impacto de EDA sobre desempenho (BLINOWSKI; OJDOWSKA; PRZYBYŁEK, 2022; CABANE; FARIAS, 2024; KAZANAVIčIUS; MAžEIKA, 2023). O recorte deste trabalho não parte da afirmação de inexistência de estudos. Busca-se produzir evidência experimental adicional em um cenário delimitado, no qual as duas variantes executam a mesma função de auditoria e são avaliadas também durante indisponibilidade e recuperação.

Formula-se, portanto, a seguinte pergunta de pesquisa:

> Quais trade-offs de desempenho e resiliência são observados entre a comunicação síncrona via REST e a comunicação assíncrona orientada a eventos no processamento de registros de auditoria?

1.2 JUSTIFICATIVA

Do ponto de vista científico, a comparação controlada permite relacionar mecanismos de comunicação a métricas observáveis, evitando recomendações baseadas somente em características teóricas. Cabane e Farias (2024) destacam a necessidade de ampliar a evidência empírica sobre o impacto da arquitetura orientada a eventos. O estudo também explicitará protocolo, variáveis, repetições e ameaças à validade, seguindo as recomendações para estudos empíricos apresentadas por Kitchenham et al. (2002).

Do ponto de vista prático, arquitetos precisam decidir se o desacoplamento oferecido por mensageria compensa sua complexidade operacional. Registros de auditoria podem ser produzidos em rajadas e precisam permanecer disponíveis para investigação, mas normalmente não integram o resultado funcional que deve ser exibido imediatamente ao usuário. Perda, duplicidade ou atraso excessivo reduzem a correspondência entre a

execução e a evidência registrada, comprometendo o valor da trilha de auditoria (AMIR-MOHAMMADIAN; CHONG; SKALKA, 2016).

1.3 OBJETIVOS

1.3.1 Objetivo geral

Comparar experimentalmente o desempenho e a resiliência de uma implementação síncrona baseada em REST e de uma implementação assíncrona orientada a eventos no processamento de registros de auditoria.

1.3.2 Objetivos específicos

a) definir um modelo comum de registro de auditoria e uma regra de processamento equivalente;

b) implementar uma variante síncrona baseada em API REST e PostgreSQL;

c) implementar uma variante assíncrona baseada em Apache Kafka, consumidor e PostgreSQL;

d) definir cenários controlados de carga, indisponibilidade e recuperação;

e) coletar latência, throughput, taxa de erro, uso de recursos, destinos dos eventos, duplicidade e tempo de recuperação;

f) analisar os trade-offs observados sem pressupor superioridade de uma alternativa.

1.4 HIPÓTESES DE TRABALHO

As hipóteses foram formuladas antes da campanha e não pressupõem sua confirmação. Esperava-se menor complexidade e menor custo básico da variante síncrona em cargas reduzidas. Esperava-se também que a variante assíncrona reduzisse a latência de aceitação e retivesse eventos durante a indisponibilidade do consumidor, com contrapartida de recursos e diferença entre aceitação e conclusão. O experimento operacionaliza custo básico como CPU e memória do caminho de processamento; custo financeiro e esforço de manutenção não foram medidos. O Capítulo 5 confronta essas expectativas com os dados e distingue confirmação descritiva, refutação e aspectos inconclusivos.

1.5 DELIMITAÇÃO

O estudo limita-se às implementações REST e Kafka construídas. Não compara RabbitMQ, gRPC ou GraphQL. Não fazem parte do escopo Kubernetes, Debezium, Event Sourcing, CQRS, Saga, replicação geográfica ou integrações externas. O ambiente foi

local e conteinerizado, com eventos sintéticos. As conclusões restringem-se às versões, configurações, cargas e recursos documentados.

1.6 ORGANIZAÇÃO DO TRABALHO

O Capítulo 2 apresenta a revisão bibliográfica. O Capítulo 3 descreve os materiais e o método experimental. O Capítulo 4 documenta o protótipo e seus diagramas. O Capítulo 5 organiza a análise pelos objetivos específicos, separando validação, campanha definitiva e qualificação parcial. O Capítulo 6 responde à pergunta de pesquisa e delimita recomendações e trabalhos futuros.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo fundamenta as decisões que serão comparadas no experimento. A exposição distingue quatro níveis frequentemente tratados como equivalentes: HTTP é um protocolo de aplicação; REST é um estilo arquitetural; arquitetura orientada a eventos (EDA) organiza a colaboração entre componentes a partir da produção e reação a eventos; e Apache Kafka é uma plataforma que pode ser utilizada para implementar fluxos assíncronos (FIELDING; TAYLOR, 2002; KARABEY AKSAKALLI et al., 2021; KREPS; NARKHEDE; RAO, 2011). A distinção é necessária porque os efeitos observados não decorrem apenas da tecnologia, mas também das garantias adotadas pela aplicação, do momento em que uma operação é considerada concluída e do modo como falhas e repetições são tratadas. REST e EDA não são categorias mutuamente exclusivas: neste protótipo, ambas as variantes recebem requisições HTTP, e a diferença está no processamento imediato ou intermediado por mensageria.

2.1 REGISTROS E TRILHAS DE AUDITORIA

Registros de auditoria descrevem ocorrências atribuíveis em um sistema e apoiam reconstrução cronológica, responsabilização e investigação posterior. Embora possam compartilhar infraestrutura com logs operacionais e de depuração, sua finalidade é diferente: um log de depuração auxilia o desenvolvimento, enquanto uma trilha de auditoria precisa conservar evidência coerente com a execução que pretende representar. Amir-Mohammadian, Chong e Skalka (2016) formalizam essa relação por propriedades de correção, completude e solidez do registro, mostrando que a análise posterior perde validade quando a instrumentação não corresponde adequadamente ao comportamento executado.

No protótipo, cada evento contém UUID, tipo do evento, tipo e identificador da entidade, ator, origem, instante e conteúdo contextual. O UUID permite correlacionar emissão, aceitação, consumo e persistência. Essa correlação também viabiliza verificar perda e duplicidade sem depender apenas dos totais agregados. O conteúdo é serializado de maneira determinística antes do cálculo de SHA-256. A representação denominada canônica neste trabalho ordena as propriedades do JSON e utiliza regras fixas de serialização no núcleo compartilhado; não se declara equivalência com um padrão universal de canonicalização. O mecanismo identifica conflitos entre o conteúdo recebido e o hash armazenado para um UUID. Não comprova a autoria do evento nem protege contra a alteração conjunta do registro e de seu hash por alguém com acesso ao banco.

Registros de auditoria são adequados ao experimento porque apresentam uma função pequena e repetível: receber, validar, identificar e persistir um fato. Essa simplicidade permite manter equivalência funcional entre as variantes. Em uso real, entretanto, a escolha arquitetural afeta o significado da confirmação. Uma resposta síncrona pode

informar ao sistema de origem que o registro já está persistido. Em uma alternativa assíncrona, a aceitação pode ocorrer antes da persistência, exigindo mecanismos adicionais para acompanhar backlog, falhas e conclusão posterior.

2.2 COMUNICAÇÃO EM SISTEMAS DISTRIBUÍDOS

Uma chamada entre processos não possui as mesmas garantias de uma chamada local. Rede, processo remoto e armazenamento podem falhar de modo independente, produzindo falhas parciais. O cliente pode receber timeout sem saber se a operação não iniciou, está em execução ou foi concluída antes de a resposta se perder. Por essa razão, repetição, timeout, identificação e observabilidade integram a semântica prática de uma operação distribuída.

Comunicação síncrona e assíncrona alteram onde essa incerteza aparece. Na interação síncrona, o cliente permanece temporalmente acoplado ao serviço e normalmente interpreta a resposta como conclusão do trabalho acordado. Na interação assíncrona, produtor e consumidor não precisam estar ativos no mesmo instante, mas a aplicação passa a distinguir aceitação, armazenamento intermediário e conclusão. O desacoplamento temporal reduz dependência simultânea, porém introduz estados intermediários que precisam ser operados e observados (KARABEY AKSAKALLI et al., 2021; HELLAND, 2007).

2.2.1 HTTP, REST e comunicação síncrona

REST é um estilo arquitetural composto por restrições como cliente-servidor, ausência de estado de sessão no servidor, possibilidade de cache, interface uniforme e sistema em camadas (FIELDING; TAYLOR, 2002). Portanto, uma API que utiliza HTTP e JSON não é automaticamente uma implementação completa de REST. Pautasso, Zimmermann e Leymann (2008) mostram que a escolha de serviços REST envolve decisões arquiteturais e consequências próprias, não apenas a seleção de um formato de mensagem. Neste estudo, a expressão “variante REST” identifica uma API orientada ao recurso de registro, utilizando HTTP de forma síncrona; a comparação não pretende avaliar individualmente todas as restrições do estilo.

Na restrição *stateless*, cada requisição deve conter as informações necessárias para ser compreendida, sem depender de contexto de sessão mantido entre requisições no servidor. Segundo Fielding e Taylor (2002), essa restrição melhora a visibilidade das interações, favorece recuperação de falhas parciais e facilita a liberação de recursos entre requisições. A ausência de estado conversacional não significa ausência de dados persistidos: o servidor pode consultar bancos, mas não deve depender de uma sessão implícita para interpretar a requisição.

Os autores também apresentam limitações: repetir contexto aumenta o volume transferido, enquanto manter o estado da aplicação no cliente reduz o controle do servidor sobre a sequência de interação (FIELDING; TAYLOR, 2002). Aplicado ao protótipo, isso

significa que a ausência de sessão não elimina gargalos no processamento ou no banco. Uma API sem sessão ainda pode ficar indisponível se todas as requisições dependerem do mesmo armazenamento.

No fluxo estudado, o cliente envia `POST /audit`; a API valida o contrato, calcula o hash, persiste e somente então responde `201 Created`. Essa definição oferece uma pós-condição direta: ao receber sucesso, o cliente sabe que existe um registro persistido. Em contrapartida, validação, processamento e banco integram o caminho crítico. A latência do banco é incorporada à latência observada pelo cliente, conexões permanecem ocupadas e a indisponibilidade de uma dependência impede a conclusão. Timeouts e novas tentativas também exigem idempotência, pois a ausência de resposta não demonstra que o efeito não ocorreu.

Pautasso, Zimmermann e Leymann (2008) relacionam decisões de integração a consequências de desenvolvimento e manutenção. No recorte deste projeto, a variante síncrona possui menos componentes a operar, enquanto a assíncrona acrescenta broker e consumidor. Essa diferença permite discutir complexidade operacional, mas não demonstra, por si só, menor custo financeiro. O monitoramento deverá acompanhar requisições, duração, respostas HTTP, timeouts e uso de recursos. Escalar a API, sem ampliar a capacidade da dependência saturada, pode apenas deslocar ou intensificar a fila de espera.

2.2.2 Mensageria e comunicação assíncrona

Mensageria introduz produtor, intermediário e consumidor. O produtor registra uma mensagem no broker sem executar diretamente a lógica do consumidor. A retenção permite que produtor e consumidor operem em ritmos distintos e que o processamento seja retomado após indisponibilidade. Esse mecanismo pode absorver rajadas, mas não elimina o trabalho: quando a taxa de entrada supera a taxa de processamento, forma-se backlog, aumentando a latência de conclusão e a necessidade de capacidade posterior.

As expressões *at-most-once*, *at-least-once* e *exactly-once* precisam ser associadas ao limite em que são garantidas. A primeira admite ausência de entrega; a segunda admite reentrega; a terceira exige explicitar qual efeito é realizado uma única vez. Kafka organiza o consumo por posições no log, enquanto o efeito produzido em um banco externo constitui outra fronteira de consistência (KREPS; NARKHEDE; RAO, 2011; HELLAND, 2007). Se o consumidor persistir no PostgreSQL e falhar antes de confirmar o offset, a mensagem poderá ser entregue novamente. O projeto busca entrega pelo menos uma vez e efeito idempotente para mensagens confirmadas no broker: a restrição única sobre `event_id` impede uma segunda linha, e o hash diferencia reentrega legítima de conteúdo conflitante. A garantia ponta a ponta depende também de confirmação de publicação, retenção e retomada do consumidor. A implementação e as verificações funcionais desses mecanismos, incluindo o encaminhamento de falhas terminais à DLQ antes da confirmação de origem, são apresentadas na Seção 4.6.3; essas verificações não demonstram garantia universal sob

quaisquer falhas.

Idempotência possui desafios práticos. O identificador precisa ser estável desde a origem; a verificação e o efeito devem estar protegidos contra concorrência; a política para o mesmo identificador com conteúdo diferente deve ser explícita; e a retenção da informação de deduplicação precisa cobrir o período de reentrega. Apenas consultar e depois inserir não é suficiente sob concorrência, pois dois consumidores podem observar ausência simultaneamente. Uma restrição única no banco torna a decisão atômica no ponto de persistência.

O uso de mensageria também muda manutenção e operação. O contrato do evento passa a ser uma dependência semântica entre produtor e consumidores. Alterações incompatíveis podem permanecer no tópico e falhar somente quando consumidas. Backlog, idade da mensagem, reentregas, registros não processáveis, estado dos grupos e diferença entre offsets produzidos e consumidos tornam-se sinais operacionais relevantes. Em compensação, a retenção permite recuperar consumidores e adicionar novos leitores sem bloquear o produtor.

2.3 ARQUITETURA ORIENTADA A EVENTOS

Um evento representa um fato relevante que já ocorreu, enquanto um comando expressa uma solicitação para que algo seja feito. Em EDA, componentes publicam eventos e outros componentes reagem a eles; os padrões de comunicação discutidos por Karabey Aksakalli et al. (2021) ajudam a distinguir essa colaboração de uma invocação direta. Neste trabalho, o evento representa uma ocorrência da aplicação cliente, e a ação do consumidor é registrar essa ocorrência. Kafka não define sozinho a arquitetura: contratos, responsabilidades, tratamento de falhas e evolução precisam ser organizados em torno dos eventos.

Entre as vantagens potenciais estão desacoplamento temporal, extensão por novos consumidores e isolamento de rajadas. Um produtor pode continuar aceitando eventos enquanto um consumidor está temporariamente indisponível, desde que o broker permaneça disponível e haja capacidade de retenção. Novos consumidores podem reutilizar o fluxo sem modificar a API produtora. Esses benefícios são condicionais: contratos continuam acoplando semanticamente os participantes, e o atraso entre produção e consumo cria consistência eventual.

As limitações aparecem principalmente na compreensão do estado global. Uma resposta de aceitação não representa a conclusão do processo; falhas podem ser descobertas depois que o cliente encerrou a interação; e a ordem entre eventos depende da estratégia de particionamento. Laigner et al. (2021) observaram que consistência, evolução de esquemas e interleaving de fluxos transferem responsabilidades para a camada de aplicação. O diagnóstico requer identificadores de correlação e combinação de logs, métricas e rastreamento. Em entrevistas com 28 profissionais, Niedermaier et al. (2019)

identificaram observabilidade e monitoramento como desafios centrais de sistemas distribuídos e destacaram a necessidade de estratégia, responsabilidades e correlação das evidências operacionais.

Em termos práticos, EDA pode ser apropriada para integração desacoplada, propagação de mudanças, telemetria e trabalhos que toleram conclusão posterior. Ela é menos adequada quando o usuário precisa de confirmação imediata do efeito ou quando a complexidade operacional não se justifica. Custos incluem infraestrutura do broker, armazenamento e tráfego, instrumentação, operação de consumidores, gestão de contratos e capacitação da equipe. Escalar consumidores aumenta capacidade apenas até limites impostos por partições, banco de dados e trabalho por mensagem.

2.4 APACHE KAFKA

Kafka é uma plataforma de mensageria baseada em log distribuído. Produtores escrevem registros em tópicos, os tópicos são divididos em partições e consumidores controlam posições identificadas por offsets. A retenção independente da leitura permite que consumidores processem o fluxo em ritmos distintos e, dentro do período configurado, retomem posições anteriores (KREPS; NARKHEDE; RAO, 2011).

O artigo de Kreps, Narkhede e Rao (2011) fundamenta o desenho original, mas não serve como comprovação das funcionalidades de versões posteriores. Seus resultados de desempenho pertencem à implementação e ao ambiente de 2011. Neste estudo, parâmetros do cliente, confirmação da publicação e comportamento sob falhas dependem da configuração efetivamente executada e deverão ser verificados no protótipo.

A ordenação é garantida dentro de uma partição, não de todo o tópico. Usar a mesma chave direciona eventos relacionados para a mesma partição quando o particionamento permanece compatível, preservando a ordem relativa desses eventos. Entretanto, aumentar o número de partições, alterar chaves ou combinar eventos de entidades distintas pode mudar a ordem global percebida. Há também um compromisso entre ordem e paralelismo: uma única partição oferece ordem total no tópico, mas limita o número de consumidores ativos no mesmo grupo.

Offsets representam a posição de consumo, e sua confirmação não é equivalente à persistência do efeito em um banco externo. Confirmar antes do efeito cria risco de perda lógica; confirmar depois permite reentrega. Uma transação que abranja Kafka e PostgreSQL não é fornecida automaticamente. Neste estudo, o consumidor persiste primeiro e confirma depois. A unicidade por UUID torna a reexecução segura quanto à duplicidade de linhas, embora continue necessário contabilizar reentregas e distinguir conflitos.

Outro ponto é a pressão de retorno. Kafka permite acumular mensagens quando consumidores ficam lentos, mas o backlog não pode crescer indefinidamente. Retenção, capacidade de disco, tempo máximo aceitável até a conclusão e velocidade de drenagem precisam ser monitorados. No experimento, a quantidade de mensagens pendentes e o

tempo de recuperação após a retomada do consumidor representarão esse comportamento.

2.5 DESEMPENHO, RESILIÊNCIA E OBSERVABILIDADE

Latência é um intervalo entre marcos definidos, não uma propriedade única. Em ambas as variantes, a latência HTTP corresponde ao envio até a resposta, e a latência de conclusão utiliza o envio até o mesmo marco registrado pelo repositório após o commit no PostgreSQL. A resposta síncrona sucede a transação; a assíncrona confirma o aceite no broker. Comparar o `202` diretamente com o `201` responderia apenas qual API devolve controle mais cedo, não qual fluxo conclui primeiro. O Capítulo 3 especifica relógios, resolução e conciliação desses marcos.

Média pode ocultar caudas. Por isso, serão apresentados mediana e percentis p95 e p99, que indicam limites abaixo dos quais se encontram, respectivamente, 95% e 99% das observações. Esses percentis são sensíveis a aquecimento, pausas do ambiente, saturação de conexões, filas internas, coleta de lixo e contenção no banco. Serão calculados sobre amostras válidas de cada repetição, mantendo resultados individuais para mostrar dispersão.

Throughput é a quantidade de registros efetivamente concluídos por unidade de tempo. Na comparação principal, conclusão será persistência, evitando contar apenas respostas `202`. A taxa oferecida pelo gerador será registrada separadamente da taxa concluída. Se a entrada superar a capacidade, o throughput pode permanecer estável enquanto backlog e latência aumentam; por isso, as três medidas precisam ser interpretadas em conjunto.

Taxa de erro será calculada como operações sem o resultado esperado divididas pelas operações tentadas. Erros HTTP, falhas de publicação, falhas de consumo e conflitos de integridade serão separados, pois possuem significados distintos. CPU e memória ajudam a interpretar eficiência, mas dependem do limite de recursos e da instrumentação. Na alternativa assíncrona, backlog, reentregas e tempo de drenagem completam a análise.

Neste estudo, resiliência indica a capacidade do sistema de manter sua função ou retomá-la após uma falha prevista. Durante a indisponibilidade do consumidor, a API pode continuar aceitando mensagens, mas isso não significa que o fluxo completo continue concluindo registros. Serão observados disponibilidade da entrada, quantidade aceita, quantidade persistida, perda, duplicidade, backlog máximo e tempo necessário para retornar ao estado estável.

Observabilidade afeta a própria capacidade de validar o experimento. Cada evento terá UUID propagado nos logs e armazenado no banco. Logs registrarão marcos de recebimento, publicação, consumo, tentativa de persistência e confirmação. Essa correlação permite calcular a latência de conclusão e investigar valores extremos. Em sistemas distribuídos, a observabilidade auxilia a reconstrução de interações e o diagnóstico, mas exige decisões técnicas e organizacionais para lidar com a complexidade e a dinâmica dos

componentes (NIEDERMAIER et al., 2019).

2.6 TRABALHOS RELACIONADOS

Cabane e Farias (2024) compararam uma aplicação implementada como monólito e como arquitetura orientada a eventos, utilizando medidas de recursos, tempo de resposta, throughput e tráfego. No ambiente estudado, a alternativa monolítica utilizou menos recursos e respondeu mais rapidamente. Esse resultado não estabelece superioridade universal. Para este TCC, interessa o cuidado em comparar a mesma função com múltiplas métricas. O recorte proposto concentra-se em duas configurações de registro e acrescenta a distinção entre aceitação e conclusão. Como Kafka e consumidor também alteram a topologia e o uso de recursos, os resultados caracterizarão essas configurações concretas, sem isolar um efeito universal de REST ou EDA.

Kazanavičius e Mažeika (2023) avaliaram REST, RabbitMQ, Kafka, gRPC e GraphQL durante decomposição de monólitos. A amplitude permite discutir critérios de seleção, mas também distribui a análise entre cinco mecanismos e um contexto de migração. O estudo sustenta que a escolha depende dos requisitos e não apenas de desempenho bruto. O presente trabalho reduz o número de alternativas para aprofundar os efeitos temporais e de falha de duas variantes equivalentes, sem avaliar decomposição organizacional ou migração de um monólito.

Blinowski, Ojdowska e Przybyłek (2022) implementaram quatro versões de uma aplicação, combinando monólito ou microsserviços com Java ou C#.NET, e avaliaram ambientes local e Azure. Entre os resultados, o monólito teve melhor desempenho em uma máquina, e acrescentar instâncias além de certo ponto prejudicou a aplicação. A análise explicita que ambiente e dimensionamento condicionam os benefícios da distribuição. Para transferir esses resultados ao presente problema, seria necessário considerar diferenças de tecnologia, carga e topologia. O TCC mantém linguagem, contrato, processamento e banco comuns às variantes, mas reconhece o broker e o consumidor como componentes adicionais da configuração assíncrona.

Karabey Aksakalli et al. (2021) revisaram 239 publicações e selecionaram 38 estudos primários, identificando três abordagens de implantação, sete padrões de comunicação, oito desafios de implantação e seis desafios de comunicação. A revisão demonstra que implantação e comunicação são decisões igualmente relevantes e que padrões diferentes atendem contextos distintos. Por ser um estudo secundário, não fornece uma medição direta do comportamento de REST e Kafka sob a mesma carga; ele apoia a seleção dos conceitos e dos desafios que precisam ser observados experimentalmente.

Laigner et al. (2021) combinaram literatura, inspeção de aplicações abertas e consulta a mais de 120 profissionais e pesquisadores para caracterizar gestão de dados em microsserviços. Foram relatados desafios de consistência eventual, evolução de esquemas, interleaving de eventos, replicação e validação de restrições na aplicação. O estudo explica

por que idempotência e integridade não podem ser tratadas apenas como propriedades do broker. Entretanto, seu objetivo é caracterizar práticas de dados, e não comparar desempenho e recuperação de dois fluxos equivalentes.

As bases conceituais também apresentam limites de aplicação. Helland (2007) é um artigo de posição publicado no CIDR: discute padrões de coordenação e mensagens diante da ausência de transações globais, sem constituir um benchmark de REST e Kafka. Amir-Mohammadian, Chong e Skalka (2016) investigam a correção da instrumentação de auditoria e apresentam uma aplicação ao OpenMRS; o presente protótipo não reproduz sua prova formal. Já Niedermaier et al. (2019) oferecem evidência qualitativa de entrevistas, útil para selecionar preocupações de observabilidade, mas insuficiente para estimar latência ou capacidade. A natureza de cada contribuição determina como ela é utilizada nesta pesquisa.

Os estudos convergem em três aspectos: distribuição adiciona custos; resultados dependem do contexto; e garantias de dados e operação não surgem automaticamente da tecnologia. Diferem quanto ao objeto, às alternativas e às métricas. No conjunto analisado, a oportunidade específica deste trabalho é comparar uma confirmação síncrona após persistência com uma aceitação assíncrona seguida de conclusão posterior, usando o mesmo contrato e armazenamento, e observar não apenas carga normal, mas indisponibilidade e drenagem de backlog. Essa lacuna é apresentada como recorte da busca e dos estudos selecionados, não como afirmação de inexistência absoluta de trabalhos.

2.7 SÍNTESE

A variante síncrona escolhida responde após persistir o registro, incorporando as dependências necessárias ao efeito em seu caminho crítico. A ausência de sessão favorece certas propriedades de REST, mas não define sozinha o momento da confirmação. Na variante assíncrona, a retenção em Kafka permite separar produção e consumo e acomodar diferenças temporárias de ritmo, exigindo acompanhamento de backlog, reentrega, ordem e conclusão posterior. O experimento será estruturado para medir essas duas configurações, manter a função de registro equivalente e interpretar conjuntamente latência, recursos e garantias de processamento.

3 METODOLOGIA

A pesquisa compreende desenvolvimento, verificação funcional, calibração dos instrumentos e comparação experimental. A verificação do software identifica se cada mecanismo funciona nas condições testadas; a comparação exige o conjunto de repetições e controles especificado neste capítulo. Os registros da primeira etapa não serão utilizados como substitutos da coleta definitiva.

3.1 TIPO DE PESQUISA

3.1.1 Natureza da pesquisa

A pesquisa é aplicada porque constrói um protótipo para apoiar decisões de engenharia de software. O procedimento principal é experimental: duas configurações de comunicação foram submetidas a cargas e falhas controladas. Desempenho e recuperação dependem de condições de execução e, portanto, não podem ser estabelecidos apenas pela descrição das tecnologias. A definição prévia de variáveis e critérios de análise segue as recomendações de Kitchenham et al. (2002).

3.1.2 Abordagem metodológica

A abordagem é quantitativa. Latência, throughput, erros, recursos, backlog e recuperação são expressos por medidas observáveis. A interpretação relaciona os números aos mecanismos executados e às limitações do ambiente. A unidade experimental é a execução planejada, com elegibilidade por métrica explicitada na A1; os eventos de uma mesma execução não são repetições experimentais independentes.

A relação com os objetivos específicos organiza o procedimento. Contrato e regras comuns estabelecem equivalência funcional; APIs, consumidor e repositório materializam as variantes; carga, instrumentação e injeção de falha operacionalizam os cenários; consolidação e análise dos dados permitem discutir os trade-offs. A equivalência funcional controla a regra de processamento, mas não elimina diferenças de topologia: Kafka e consumidor adicionais integram o custo observado da variante assíncrona.

3.1.3 Procedimentos técnicos

Os procedimentos incluem estudo de artigos científicos, desenvolvimento incremental, testes unitários e de integração, calibração e experimento controlado. O levantamento bibliográfico fundamenta conceitos e situa os estudos relacionados; a resposta à pergunta de pesquisa dependerá de medições no protótipo.

3.2 PROCEDIMENTOS METODOLÓGICOS

3.2.1 Métodos de coleta de informações

A fundamentação utiliza exclusivamente artigos científicos publicados em periódicos e anais de conferências ou workshops. Os tipos de contribuição e as limitações dos estudos são discutidos no Capítulo 2.

Na coleta computacional, cada evento possui UUID e instante de ocorrência definidos antes do envio. O conjunto completo de campos é preservado em reentregas. Um identificador UUID de execução, `run_id`, é enviado no cabeçalho `X-Run-ID` e propagado ao consumidor por cabeçalho Kafka. Esse metadado identifica a coleta, sem alterar o conteúdo usado no hash.

O gerador de carga com k6 e o analisador independente em Java foram implementados no diretório de experimentação do projeto. A calibração inicial foi registrada em 26 de outubro de 2026; em 27 de outubro, a revisão reduzida R2 passou nas nove validações adicionais e foi congelada antes de qualquer coleta definitiva. Esses instrumentos não são módulos das APIs nem substituem seu processamento. O executor registra fases separadas de aquecimento e medição e utiliza um projeto Docker Compose isolado. As aplicações Java emitem marcos JSON de etapa, UUID, execução, resultado e tempo. O marco `commit_completed` é emitido pelo repositório comum depois do retorno da transação, tanto na variante síncrona quanto no consumidor. Uma repetição idempotente é identificada como `duplicate` e não representa uma nova linha persistida.

A coleta mantém três instantes distintos: ocorrência do fato, conclusão observada pela aplicação depois do commit e recebimento da resposta HTTP. O campo `persisted_at` é preenchido durante a inserção no PostgreSQL; não representa o término exato do commit. A consulta `GET /audit/{event_id}` verifica a visibilidade da linha, mas seu instante de resposta também não substitui o marco pós-commit.

O tempo de requisição é observado pelo gerador de carga. Na correlação entre processos, utiliza-se uma referência UTC comum; resolução e consistência foram verificadas no piloto. A presença de timestamps em nanossegundos nos logs não aumenta por si só a resolução da medida de ponta a ponta. Relógios monotônicos são usados nas durações locais. Valores incompatíveis são sinalizados como falha de instrumentação, sem truncamento silencioso; essas verificações não comprovam ausência de todo desvio entre relógios.

Na execução experimental, estatísticas Docker registram CPU e memória por contêiner. Offsets do tópico e do grupo permitem observar o lag; consultas ao PostgreSQL verificam existência, hash e unicidade. O executor preserva arquivos brutos, configuração, versões, identificação das imagens e checksums. Esses instrumentos já foram implementados e validados no piloto; a campanha definitiva, distinta das verificações funcionais da Seção 4.6, fornece as repetições necessárias à análise comparativa.

3.2.2 Métodos de análise das informações

A análise parte dos dados brutos preservados, correlacionados por execução e UUID. Para cada repetição são calculados quantidade oferecida, aceita e concluída; média, mediana, desvio-padrão e percentis p95 e p99 das latências; throughput concluído; erros por categoria; backlog máximo; reentregas e recuperação. A taxa oferecida efetiva e as iterações que o gerador não conseguiu iniciar são apresentadas separadamente. Na consolidação HTTP válido, percentis são calculados por posto mais próximo sobre as amostras de cada execução; preservam-se também os resumos originais do analisador e do k6, cujos métodos de interpolação podem produzir pequenas diferenças. Não se misturam suas populações ou percentis.

Primeiro são examinados os resultados de cada repetição. Para o orçamento reduzido de cinco repetições por condição, a análise predeclarada é descritiva: valores individuais, média, mediana, desvio-padrão amostral, mínimo, máximo e diferenças pareadas entre variantes. A direção das cinco diferenças é discutida sem convertê-la em teste de superioridade ou equivalência estatística. Não foram acrescentadas execuções para obter significância ou favorecer uma hipótese. Testes com poucos eventos destinados à depuração não sustentam inferências sobre desempenho ou superioridade arquitetural.

3.3 METODOLOGIA DE DESENVOLVIMENTO DO SISTEMA

3.3.1 Modelo de desenvolvimento de software

O desenvolvimento é incremental: contrato, serialização e persistência; variante síncrona; produtor e consumidor; tratamento de falhas; instrumentação; calibração; coleta definitiva. Cada incremento mantém requisitos, código, diagramas e verificações correspondentes. Casos com doubles de bibliotecas verificam decisões locais; testes de integração utilizam Kafka e PostgreSQL reais.

3.3.2 Tecnologias utilizadas

O protótipo utiliza Java com bytecode alvo 21, Spring Boot 3.5.16, Spring JDBC, Flyway, PostgreSQL 16, Apache Kafka 3.9.1 e Docker Compose. O contrato `AuditEvent` e o repositório comuns reduzem divergências funcionais. PostgreSQL oferece transações e unicidade; Kafka oferece retenção e posições de consumo; Compose descreve a implantação local. O gerador k6, o analisador Java, a conciliação e os coletores foram implementados e calibrados. As imagens e o código instrumentador dessa calibração serão preservados na revisão reduzida; não serão reconstruídos durante as medições.

Cada execução deverá registrar sistema operacional, processador, memória, versões efetivas, identificadores das imagens, limites de CPU e memória, partições, replicação, parâmetros de timeout e retry e propriedades da carga. O commit Git da implementação Java e checksums dos arquivos comporão o manifesto. Credenciais permanecerão na

configuração privada do ambiente e não integrarão relatórios, capturas ou dados experimentais.

3.3.3 Arquitetura do sistema

As variantes compartilham `AuditEvent`, serialização determinística, SHA-256 e repositório PostgreSQL. Na síncrona, a resposta 201 sucede a transação ou o reconhecimento de duplicata. Na assíncrona, a API aguarda confirmação individual de publicação com `acks=all`; o consumidor processa posteriormente. A resposta 202 confirma essa publicação, sem atestar persistência no PostgreSQL. Um timeout de publicação é classificado como resultado incerto, pois a mensagem pode ter sido entregue sem que a confirmação chegasse ao cliente.

O consumidor desabilita commit automático de offsets. Após inserção ou reentrega idêntica, confirma a posição consumida. Erros transitórios de persistência recebem tentativas limitadas com espera exponencial; mensagens inválidas, conflitos ou tentativas esgotadas são encaminhados à DLQ. A posição de origem somente é confirmada depois do ACK da DLQ. Se a publicação na DLQ ou a confirmação da posição falhar, a sessão é retomada sem avançar além da mensagem não resolvida. Isso permite reentrega e exige idempotência; não estabelece uma transação única entre Kafka e PostgreSQL.

A topologia local contém um broker e uma réplica por tópico. A confirmação configurada não demonstra tolerância à perda física do broker nem alta disponibilidade de cluster. Limites de recursos do ambiente de desenvolvimento estão explícitos no Compose; sua adequação às configurações definitivas deverá ser verificada no piloto.

3.4 PROCEDIMENTOS DE TESTE E VALIDAÇÃO

3.4.1 Tipos de testes aplicados

Testes unitários Java verificam contrato, serialização, hash, decisões de confirmação, retry, DLQ e respostas HTTP. O fluxo de integração contínua executou *smokes* com PostgreSQL e Kafka reais para os caminhos normais e para uma interrupção do banco. A cobertura desses *smokes* é delimitada na Seção 4.6.3; testes adicionais de integração, falhas e carga ainda serão necessários. Verificações funcionais documentam o software e não respondem isoladamente à pergunta experimental.

O piloto verificou geração de carga, coleta, resolução temporal, limites e duração. Correções instrumentais pertencem a essa fase. O protocolo foi congelado antes da coleta definitiva; a revisão R1 rejeitada permanece preservada. A emenda A1 posterior, detalhada adiante, alterou somente análise offline e controle de admissão, sem mudar instrumentos, imagens, parâmetros ou repetir UUIDs.

3.4.2 Cenários de teste

C1 utilizará carga baixa para observar correção sem saturação. C2 aplicará níveis graduais para avaliar latência e capacidade. C3 produzirá uma rajada acima da taxa estável. C4 introduzirá indisponibilidade durante entrada ativa. C5 acompanhará a retomada e a drenagem do backlog.

A calibração inicial confirmou uma referência comum de 48 eventos por segundo durante cinco minutos de medição em cada variante, após aquecimento. Essa taxa é uma referência estável observada, não a capacidade máxima. C2 utilizará 12, 24, 36 e 48 eventos por segundo, correspondentes a 25%, 50%, 75% e 100% da referência; C1 compartilha a condição de 12 eventos por segundo, sem duplicar sua contagem amostral. C3 utilizará a rajada 48–96–48, com duas transições de um segundo. Cada par receberá a mesma taxa absoluta e conteúdo determinístico correspondente; serão informadas diferenças entre volume programado e efetivamente enviado.

Em C4, a falha escolhida é a interrupção do PostgreSQL nas duas variantes, para comparar a reação à mesma dependência indisponível. A API síncrona pode retornar 503, enquanto a assíncrona ainda pode confirmar publicação com 202 se Kafka estiver disponível; esse aceite não garante persistência posterior. Se o orçamento de tentativas do consumidor se esgotar, a mensagem segue à DLQ e não há reprocessamento automático depois da volta do banco. O piloto fixará duração da interrupção, prazo de drenagem e critério de estabilidade de C5.

Antes de qualquer coleta definitiva, foi autorizada uma revisão do orçamento experimental: o protocolo anterior, de dez repetições e 300 segundos de medição, foi preservado como versão 1, sem reclassificar calibrações ou demonstrações como resultados definitivos. A versão 2 manterá as 13 combinações de condição e variante, com cinco repetições por combinação, totalizando 65 execuções. Cada execução terá 60 segundos de aquecimento excluídos da análise, intervalo de 12 segundos, 180 segundos de medição e 90 segundos de observação posterior. Os cenários de falha utilizarão 12 eventos por segundo e interrupção solicitada de 40 segundos, iniciada aproximadamente aos 60 segundos da medição; os instantes efetivos serão registrados. Haverá 128 usuários virtuais disponíveis nas condições normais e 176 nas falhas, sem pressupor simultaneidade constante.

A redução é uma escolha explícita de orçamento, não um cálculo de poder estatístico nem demonstração de equivalência com o tamanho amostral anterior. Cinco repetições reduzem a precisão entre execuções e a janela menor pode deixar de observar efeitos tardios. A análise apresentará valores por repetição, média, mediana, desvio-padrão, extremos e diferenças pareadas, sem tratar eventos individuais como repetições independentes ou converter ausência de diferença clara em equivalência. Resultados mistos ou insuficientes permanecerão inconclusivos.

Na revisão R2, somente a sonda periódica de saúde do broker experimental foi substituída: em vez de iniciar uma JVM administrativa para listar tópicos a cada dez

segundos, uma conexão TCP local verifica a disponibilidade de transporte. Essa sonda leve não comprova funcionamento semântico do Kafka; criação efetiva de tópicos e verificação da API continuam obrigatórias antes da carga. O objetivo é não misturar repetidas inicializações da ferramenta administrativa ao processamento medido do broker. Imagens, quotas, regras de negócio, referência de 48 eventos por segundo e critérios de elegibilidade são mantidos. O ensaio R1 que motivou a investigação permanece identificado e não será aprovado retrospectivamente ou contado como definitivo.

O congelamento da versão 2 foi condicionado a nove validações identificadas separadamente: modelo comum nas duas variantes; referência de 48 eventos por segundo e rajada nas duas; interrupção do PostgreSQL nas duas; e interrupção do consumidor apenas na assíncrona. As sete validações de carga utilizaram a janela e os limites propostos para os definitivos. Exigiram integridade instrumental, margem dos coletores, elegibilidade da referência e recuperação sustentada durante entrada ativa nos cenários de falha. Os nove casos foram aprovados e auditados em 27 de outubro de 2026, com preservação dos dados e da tentativa R1 não elegível. Não foram alongadas retrospectivamente as janelas ou afrouxados os critérios; a aprovação libera a campanha, mas não substitui suas 65 repetições.

A ordem das condições foi embaralhada pela semente registrada 20261026, em cinco blocos completos. A primeira variante foi contrabalançada globalmente; em cada condição pareada a proporção foi 3/2 ou 2/3, pois cinco repetições não permitem igualdade exata. Entre execuções, somente a tabela sintética foi reinicializada, com tópico e grupo novos; volumes, broker e caches não foram fisicamente zerados. As evidências anteriores foram exportadas e preservadas antes da próxima execução. Validações, calibrações e demonstrações não integram as 65 repetições definitivas.

3.4.3 Métricas coletadas

A duração HTTP nativa do k6, `http_req_duration`, não inclui todos os custos anteriores de conexão e bloqueio; o intervalo instrumentado entre envio e retorno é uma medida distinta. A comparação principal de resposta utilizará o p95 das respostas válidas esperadas (201 na síncrona e 202 na assíncrona, com UUID correspondente), acompanhado da quantidade de observações e das categorias de todas as respostas, inclusive erros. Na síncrona, o sucesso sucede a persistência; na assíncrona, sucede o ACK de publicação. A latência de conclusão utiliza, em ambas, o início do envio e o marco comum pós-commit, conciliado com o banco. Resposta e conclusão serão apresentadas separadamente, evitando comparar aceitação de uma variante com conclusão da outra ou interpretar erros rápidos como melhor desempenho.

Throughput concluído é a quantidade de UUIDs únicos efetivamente persistidos na janela de medição dividida pela duração dessa janela. Eventos concluídos somente durante a drenagem são contabilizados à parte. Entradas oferecidas, respostas aceitas

e linhas persistidas constituem conjuntos distintos. Uma confirmação pós-commit cujo registro de envio esteja ausente é uma inconsistência de coleta, e não uma amostra válida de latência.

Taxa de erro será discriminada em HTTP, publicação, validação, persistência, conflito e falha terminal. O lag de Kafka resulta da diferença entre posição final e posição confirmada por partição. A diferença entre eventos aceitos e persistidos constitui outro indicador e não será tratada como sinônimo de lag. Tentativas locais de retry, releituras da mesma posição Kafka e reenvios de um mesmo UUID também serão contados separadamente.

Ao fim do prazo de observação, os eventos aceitos sem persistência serão separados em falhas terminais registradas na DLQ, eventos ainda pendentes e ausências não conciliadas. Uma mensagem em DLQ foi retida para inspeção; não representa sucesso de auditoria, mas tampouco demonstra desaparecimento no transporte. A afirmação de perda exige investigar a ausência na persistência e nas evidências de retenção. Submissões de resultado incerto serão conciliadas sem serem automaticamente classificadas como aceites confirmados ou perdas.

Duplicidade persistida corresponde a mais de uma linha para o mesmo UUID e deve ser zero. Conflito corresponde ao mesmo UUID associado a conteúdo diferente. O tempo de recuperação será medido desde a retomada até o atendimento do critério de estabilidade de backlog e throughput fixado no protocolo. Uma eventual observação isolada de lag zero não substituirá esse critério científico.

3.4.4 Emenda analítica após início da campanha

Em 27 de outubro de 2026, após 37 tentativas definitivas, foi autorizada uma emenda analítica versionada, denominada A1. Sua motivação foi a coexistência, em uma coleta síncrona de interrupção do PostgreSQL, de HTTP 503 e registro persistido, mas sem marco pós-commit. A regra original invalidava globalmente a discrepância entre banco e marco. O resumo inválido, o motivo e os dados brutos permanecem preservados; a emenda não declara essa coleta integralmente válida pelo critério original. Sua aplicação é retrospectiva nas primeiras 37 tentativas e declarada antes das 28 restantes, não uma regra predeclarada antes de todos os dados.

A qualificação parcial é restrita a eventos medidos no cenário síncrono de interrupção do PostgreSQL: cliente completo, HTTP 503, conteúdo e hash corretos no banco, um marco de recebimento e um de falha por indisponibilidade do banco, sem confirmação de persistência ou commit, retry, replay, DLQ ou outra inconsistência instrumental. A mesma regra será aplicada a todo o conjunto definitivo. Dados incompletos de cliente, sucesso sem marco, marco sem registro, conteúdo incorreto, relógio inválido, coleta insuficiente, falta de folga ou casos fora desse escopo continuam bloqueios. Não há nova carga de piloto, mudança nas imagens ou instrumentos, nem repetição ou substituição da

observação.

As métricas terão elegibilidade própria. HTTP 503 continua erro, ainda que o registro apareça na exportação final; contagens finais e recursos auditados permanecem utilizáveis. A execução parcial e sua contraparte serão excluídas apenas da comparação pareada primária de latência de conclusão. Sua distribuição parcial será apresentada separadamente como sensibilidade, com quantidades de tempos observados e registros persistidos. Nenhum timestamp será imputado: o campo preenchido durante o INSERT, o horário do erro e o comando de retomada não substituem a conclusão da transação.

Para vazão, sejam c os commits observados e conciliados dentro da janela, u os registros medidos persistidos sem marco temporal e T a duração da medição. A execução parcial terá limites conservadores $[c/T, (c+u)/T]$, não um valor preenchido nem intervalo de confiança. Recuperação será descrita pelos marcos observados, sem reconstruir o primeiro commit real desconhecido. A ausência temporal na fronteira da falha não será assumida aleatória ou inofensiva por ser numericamente pequena.

O tamanho efetivo é informado por indicador. O catálogo final registra 64 coletas completas e uma parcial nos 65 UUIDs; a comparação pareada de interrupção do PostgreSQL tem quatro pares completos para latência de conclusão, embora indicadores identificados mantenham cinco observações. A análise estrita original exclui o par afetado de todos os indicadores pareados como sensibilidade, sem alterar os resumos ou repor a coleta. As conclusões permanecem limitadas pela decisão pós-hoc, pela cobertura temporal e pelo pequeno número de repetições.

3.4.5 Critérios de aceitação

O núcleo será considerado funcional nos cenários verificados quando: eventos válidos forem persistidos; entradas inválidas forem rejeitadas; 201 ocorrer após confirmação da transação; 202 depender de ACK individual; duplicatas preservarem uma linha; conflitos preservarem o registro original; retry e DLQ obedecerem à política; e offsets de origem não forem confirmados antes da persistência ou retenção na DLQ.

A coleta definitiva será aceita se configuração e snapshot forem registrados, instrumentos estiverem validados, protocolo estiver congelado, aquecimento estiver separado, repetições previstas forem identificadas, ordem estiver registrada e dados brutos permitirem conciliação. Falhas do instrumento, reinicializações externas e dados incompletos permanecem causas de invalidade sob os critérios originais; a A1 acrescenta somente a qualificação parcial e a elegibilidade por métrica explicitadas acima. Um resultado desfavorável à hipótese não invalida a execução.

3.4.6 Protocolo de execução e reprodutibilidade

O executor implementado inicia o projeto Compose isolado com imagens existentes, aguarda prontidão, confirma esquema e tópicos, registra commit e configuração,

executa aquecimento e carga, registra falha e retomada, aguarda observação posterior e exporta logs, offsets e registros com checksums. O controlador reduzido acrescenta plano predeclarado, exclusão mútua e diário de progresso, recusando sobrescrever UUIDs existentes ou iniciar uma coleta concorrente. As imagens calibradas não são reconstruídas por esse procedimento.

Cada `run_id` possui manifesto, eventos enviados, saída do gerador, logs JSON, estatísticas de recursos, offsets e exportação do banco. O controlador preserva diretórios anteriores e interrompe para investigação se houver falha de coleta ou margem insuficiente do instrumento. O protocolo e os comandos são versionados no repositório. Apenas o projeto interativo deste TCC é parado durante a medição e retomado ao final ou após saída tratável; projetos alheios não são interrompidos. A validade da revisão reduzida e os resultados definitivos serão relatados no Capítulo 5 somente após sua verificação efetiva.

3.4.7 Ameaças à validade

A validade interna é afetada por concorrência com outros processos, caches, aquecimento, virtualização, limites de recursos, coleta de telemetria e ordem dos ensaios. A mitigação inclui registrar o ambiente, utilizar configurações estáveis, reduzir processos concorrentes durante a coleta, repetir e alternar a ordem. O custo do broker adicional pertence ao objeto comparado; por isso, a pesquisa não isola universalmente um estilo arquitetural.

A validade de construto depende dos marcos temporais e da classificação dos eventos. Separar ACK, commit e resposta HTTP reduz ambiguidade. Resolução de relógios, atraso da observação e custo de emissão dos logs limitam a precisão. Lag, fila pendente, DLQ e perda são estados distintos, cujo tratamento deve ser verificável nos dados.

A validade de conclusão depende de repetições suficientes e de sua independência. Com cinco repetições, a análise examinará valores individuais, dispersão e magnitudes das diferenças pareadas, sem interpretar milhares de eventos correlacionados como milhares de experimentos ou alegar superioridade estatística. A ordem contrabalançada não elimina toda interferência ou dependência do ambiente compartilhado. Resultados de validação funcional não serão extrapolados para desempenho.

A validade externa é limitada por dados sintéticos, domínio simplificado, uma implementação, broker único, versões e ambiente local. Não serão generalizadas conclusões para todas as APIs REST ou arquiteturas EDA. A preservação do protocolo, das versões e dos dados permite replicações que investiguem essas limitações.

4 DESENVOLVIMENTO DO SISTEMA

4.1 VISÃO GERAL DO SISTEMA

O sistema consiste em uma aplicação experimental para receber e armazenar registros de auditoria, acompanhada de um ambiente de avaliação das formas de comunicação. Um registro descreve um fato ocorrido, sua origem, a entidade envolvida e o ator responsável. O processamento compreende validação, serialização determinística, cálculo de SHA-256 e persistência com unicidade por identificador. O sistema não executa a operação de negócio que originou o fato, como alterar um cadastro: apenas recebe sua representação.

O público-alvo compreende pesquisadores e estudantes que reproduzam os ensaios e profissionais que analisem os custos de comunicação síncrona e assíncrona. A aplicação cliente envia eventos; o pesquisador configura o ambiente e interpreta as evidências. Não se prevê, neste escopo, um produto corporativo de auditoria, gerenciamento de usuários, edição de registros persistidos ou integração com dados pessoais reais.

Na variante síncrona, a API HTTP valida o evento e solicita ao repositório sua persistência antes de responder. Na variante assíncrona, outra API valida o mesmo contrato e encaminha a mensagem ao produtor Kafka. Um consumidor recebe as mensagens do tópico e utiliza o mesmo processamento e repositório. PostgreSQL mantém os registros; Kafka intermedeia o fluxo assíncrono. O monitoramento e os cenários de carga constituem uma camada experimental de geração, coleta e análise, separada da função de registro.

As funcionalidades implementadas incluem recebimento HTTP, validação, hash determinístico, persistência idempotente, confirmação individual de publicação, consumo, retry limitado, DLQ, confirmação de offsets, consultas de prontidão e de registro e documentação interativa das APIs para testes manuais. A geração de carga e a consolidação quantitativa ainda precisam ser adaptadas e validadas para esta implementação Java. Os quatro wireframes descrevem a interface de operação proposta; a comparação científica definitiva depende do protocolo e das repetições previstos no Capítulo 3. A Figura 1 resume os dois caminhos operacionais implementados.

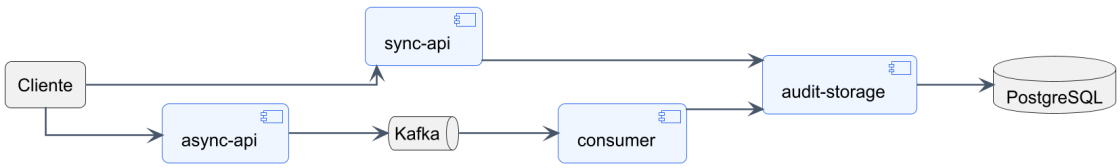


FIGURA 1 – Visão geral dos caminhos síncrono e assíncrono em Java
FONTE: Elaborado pelo autor (2026).

Nesta documentação, **implementado** indica que existe código para o comportamento descrito; as verificações disponíveis e seu alcance são apresentados na Seção 4.6.3.

Parcial indica cobertura incompleta do requisito. **Planejado** identifica um componente ou procedimento ainda não implementado. Uma funcionalidade aprovada em teste não implica validação da hipótese científica.

A resposta `202 Accepted` sucede o ACK do broker e não significa que o registro já exista no PostgreSQL. Falhas de entrega podem produzir resultado incerto, enquanto falhas de consumo podem resultar em retry ou DLQ. A presença de uma mensagem na DLQ permite inspeção, mas não equivale a processamento concluído. O broker único limita a avaliação à topologia local, sem garantia de alta disponibilidade de cluster.

Síntese. As duas rotas executam o mesmo processamento e diferem na comunicação e nas condições de confirmação. O estado de cada requisito é vinculado ao código e às evidências, preservando a distinção entre validação funcional e comparação experimental.

4.2 LEVANTAMENTO E ESPECIFICAÇÃO DE REQUISITOS

Os identificadores RF01 a RF10 são preservados para manter a rastreabilidade com a documentação anterior. RF11 a RF15 tornam explícitas as funcionalidades de avaliação e interface previstas; RF16 e RF17 detalham tratamento de falhas e consulta por UUID. A prioridade **essencial** designa uma condição necessária ao núcleo ou à validade experimental; **desejável** designa uma facilidade de uso. A Tabela 1 distingue código Java existente de instrumentos planejados; o alcance dos testes está na Seção 4.6.3.

4.2.1 Requisitos funcionais

TABELA 1 – Requisitos funcionais e critérios de aceitação

ID e estado	Descrição, prioridade e justificativa	Critério verificável de aceitação
RF01 Implementado	Receber evento em `POST /audit`. Essencial: entrada equivalente nas variantes.	As duas APIs aceitam o mesmo JSON e retornam o UUID em respostas de sucesso.
RF02 Implementado	Validar contrato. Essencial: impedir dados estruturalmente inválidos.	UUID e data informados devem ser válidos; textos respeitam os limites do dicionário; entrada inválida retorna 422, sem processamento posterior.
RF03 Implementado	Concluir registro síncrono antes da resposta. Essencial: definir a semântica síncrona.	Resposta 201 ocorre somente após commit ou identificação de duplicata idêntica; status distingue `persisted` de `duplicate`.

ID e estado	Descrição, prioridade e justificativa	Critério verificável de aceitação
RF04 Implementado	Encaminhar evento ao Kafka. Essencial: viabilizar processamento desacoplado.	Evento preserva UUID e conteúdo; chave é `entity_id`. 202 somente após ACK individual do broker com tópico, partição e offset no log. Timeout de entrega retorna 503 com resultado incerto.
RF05 Implementado	Consumir mensagens. Essencial: concluir a rota assíncrona.	Mensagem válida do tópico configurado é desserializada, validada e encaminhada ao mesmo repositório da variante síncrona.
RF06 Implementado	Gerar hash determinístico. Essencial: detectar conteúdo divergente.	O mesmo evento completo gera o mesmo SHA-256 hexadecimal de 64 caracteres; a ordenação de chaves não altera a representação serializada.
RF07 Implementado	Persistir registro. Essencial: manter efeito observável.	Linha contém os campos de `AuditEvent` e `content_hash`; `event_id` é chave primária.
RF08 Implementado	Reconhecer reentrega idêntica. Essencial: evitar efeito duplicado.	Duas submissões com UUID e conteúdo iguais mantêm uma linha e retornam status de duplicata na segunda; conteúdo divergente não substitui o original.
RF09 Implementado	Confirmar offset após persistência. Essencial: reduzir risco de perda por confirmação antecipada.	Commit automático desativado; confirmar origem somente após persistência idempotente ou ACK da DLQ. Falha de confirmação mantém possibilidade de reentrega.
RF10 Implementado	Expor `GET /health`. Essencial: verificar resposta da API.	`/health` verifica PostgreSQL na API síncrona e metadados do tópico na assíncrona; retorna 200 ou 503. `/live` verifica apenas a resposta do processo.
RF11 Planejado	Coletar métricas por execução. Essencial: responder à pergunta experimental.	Um futuro coletor deverá produzir arquivos conciliáveis por UUID e execução, com aceitação, conclusão, throughput, erros e recursos, incluindo unidade e janela. Os logs Java isolados não satisfazem esse critério.

ID e estado	Descrição, prioridade e justificativa	Critério verificável de aceitação
RF12 Parcial	Executar cenários de carga e falha. Essencial: controle experimental.	Há um *smoke* de interrupção do banco na CI. O executor definitivo ainda deverá registrar taxa, duração, semente, variante, falha e configuração em manifesto reproduzível.
RF13 Parcial	Acompanhar recuperação. Essencial: avaliar comportamento após falhas.	O *smoke* verifica retomada da saúde do banco, mas ainda falta registrar backlog, drenagem e conciliação dos eventos aceitos e persistidos.
RF14 Planejado	Consultar indicadores e comparar variantes. Desejável: facilitar inspeção.	Um relatório futuro deverá separar latência HTTP de conclusão e sinalizar ausência de dados. Não há interface ou normalizador comparativo Java implementado.
RF15 Parcial	Disponibilizar envio interativo para demonstração. Desejável: facilitar inspeção manual.	O Swagger UI único permite selecionar a variante, enviar um JSON de exemplo e consultar a resposta. O formulário próprio do wireframe, com controle explícito de UUID e instante para reenvio, permanece proposto.
RF16 Implementado	Tratar falhas e quarentena. Essencial: evitar perda por avanço prematuro.	Aplicar retry limitado com backoff; publicar erro terminal na DLQ e aguardar ACK antes de confirmar a origem.
RF17 Implementado	Consultar registro por UUID. Essencial: verificar efeito persistido.	A API síncrona retorna hash e timestamp da linha; 404 indica ausência observada e 503 indica consulta indisponível.

Fonte: Elaborado pelo autor (2026).

4.2.2 Requisitos não funcionais

A Tabela 2 relaciona propriedades de qualidade, critérios verificáveis e justificativas. Desempenho não é definido por um código HTTP: 201 e 202 não são limites de tempo. Não se impõe, antes do piloto, que a alternativa assíncrona seja mais rápida. Os critérios de medição e as condições de comparação substituem expressões não verificáveis como *tempo reduzido*.

TABELA 2 – Requisitos não funcionais

ID	Categoria e prioridade	Critério e justificativa
RNF01	Equivalência; essencial	Utilizar DTO, serialização e repositório comuns; comparar o mesmo conjunto de eventos com a mesma configuração de persistência. Evita atribuir diferenças funcionais à comunicação.
RNF02	Reprodutibilidade; essencial	Registrar imagens, dependências, commit, recursos, scripts e parâmetros. O Compose explicita limites de desenvolvimento; manifesto e checksums da coleta definitiva ainda precisam ser implementados e validados.
RNF03	Segurança; essencial	Usar dados sintéticos sem segredos; executar em ambiente isolado. O protótipo não possui autenticação nem é autorizado para exposição pública ou dados pessoais reais.
RNF04	Portabilidade; essencial	Descrever build e inicialização com Docker Compose, além dos pré-requisitos. Versões de bibliotecas são fixadas; tags de imagens deverão ter digest registrado na coleta.
RNF05	Integridade e correlação; essencial	Preservar UUID e instante de origem nos reenvios; manter hash e restrição de unicidade. Conflitos devem ser distinguíveis de duplicatas legítimas nos registros de diagnóstico.
RNF06	Desempenho; essencial	Medir latência de aceitação e de conclusão separadamente, p50/p95/p99, throughput persistido e taxa oferecida; definir tolerâncias de carga no piloto, conforme o Capítulo 3.
RNF07	Resiliência; essencial	Após falha controlada e retomada, conciliar os UUIDs dentro da janela de drenagem; zero linhas duplicadas. Eventos na DLQ exigem tratamento posterior e não retornam automaticamente ao banco.
RNF08	Observabilidade; essencial	Registrar execução, UUID, componente, etapa, instante e resultado; coletar recursos com intervalo fixado. Os marcos JSON já existem; coletores, completude e resolução temporal ainda precisam de validação para cada execução definitiva.
RNF09	Escalabilidade; desejável	Registrar partições e número de consumidores em cada configuração. Não inferir escalabilidade horizontal a partir de uma execução em broker único.

ID	Categoria e prioridade	Critério e justificativa
RNF10	Usabilidade experimental; desejável	Identificar a variante e o estado dos dados em cada tela; oferecer mensagens legíveis e unidades. Wireframes não equivalem a um painel implementado.

Fonte: Elaborado pelo autor (2026).

4.2.3 Regras de negócio

Neste contexto, regras de negócio são invariantes do processamento de auditoria e do contrato experimental, e não políticas de um sistema comercial.

RN01 — Identidade. Todo evento processado possui UUID. O DTO gera UUID se omitido; nos ensaios, o gerador deverá fornecê-lo antes do envio e preservá-lo nas novas tentativas. Novo UUID significa novo evento.

RN02 — Validação. Tipo do evento, tipo e ID da entidade, ator e origem são obrigatórios e respeitam os limites do DTO. UUID, instante e payload possuem valores padrão quando omitidos. A obrigatoriedade no banco não significa obrigatoriedade de envio desses três campos na API. Na mensagem Kafka, identidade e instante devem estar explícitos: o consumidor rejeita sua ausência para que uma reentrega não gere outro evento.

RN03 — Hash. O SHA-256 abrange todos os campos do evento validado, incluindo UUID e instante. JSON usa chaves ordenadas e separadores compactos. A regra é determinística no contexto da implementação; não se declara conformidade com um padrão externo de canonicalização.

RN04 — Idempotência. Mesmo UUID e mesmo hash preservam a linha original e produzem status `duplicate`. O cliente deve reenviar o mesmo instante; gerar outra data pode tornar o conteúdo divergente.

RN05 — Conflito. Mesmo UUID e hash diferente não podem sobrescrever a linha. A API síncrona retorna 409 com motivo `event_id_content_conflict`. No consumidor, o conflito é encaminhado à DLQ sem alterar a linha original.

RN06 — Aceitação assíncrona. O 202 não confirma persistência em PostgreSQL. Confirma publicação segundo `acks=all` e a topologia configurada. Se o ACK não chegar no prazo, a resposta 503 informa resultado incerto; o cliente deve conciliar e preservar o evento completo em eventual reenvio.

RN07 — Offset e falha. O offset de origem é confirmado após inserção, reconhecimento idempotente ou ACK da DLQ. Falhas de persistência recebem tentativas

limitadas com backoff. Se DLQ ou commit de offset não forem confirmados, a sessão é retomada sem avançar além da origem não resolvida.

RN08 — Confirmação síncrona. A API retorna 201 após persistência ou reconhecimento de reentrega, distinguindo-os pelo campo status. Em falha de persistência não deve declarar sucesso.

RN09 — Particionamento. A chave enviada ao produtor é `entity_id`. A ordem relativa depende da manutenção da chave, do particionamento e da sequência produzida; não se promete ordenação global entre partições.

RN10 — Evidência experimental. As métricas definitivas deverão identificar execução, variante, unidade e janela. Eventos gerados, aceitos e persistidos são conjuntos distintos. O marco pós-commit comum define conclusão; instante de ocorrência e timestamp de inserção não o substituem.

RN11 — Quarentena. Mensagens inválidas, conflitos e tentativas esgotadas são retidos na DLQ. A chave de origem usa tópico, partição e offset. Se a DLQ for confirmada e o commit de origem falhar, podem existir cópias na DLQ; a conciliação futura deverá usar essa chave. Não há replay automático de mensagens em quarentena.

Síntese. Os requisitos indicam o comportamento esperado, o critério de verificação e o estado de implementação. As regras tornam explícitos os limites do aceite assíncrono, da identidade do evento e da recuperação.

4.3 MODELAGEM UML

As vistas representam o contrato, as responsabilidades e os caminhos de execução. A fronteira operacional é a implementação Java, com duas APIs Spring Boot e um consumidor independente; Kafka e PostgreSQL são sistemas externos de suporte. Na visão de implantação, esses serviços integram a infraestrutura do ambiente experimental. As bibliotecas Maven compartilhadas não são atores nem processos independentes.

4.3.1 Diagrama de casos de uso

A aplicação cliente inicia UC01 e UC02; Kafka fornece mensagens a UC03; o pesquisador consulta prontidão e registros em UC04 e UC07. A Figura 2 apresenta essas interações. A Figura 3 reúne os casos planejados de avaliação quantitativa, cujo executor ainda não integra o projeto Java.

4.3.1.1 UC01 — Registrar auditoria sincronamente

Objetivo e atores: persistir evento antes da resposta; aplicação cliente e PostgreSQL. **Pré-condições:** API inicializada e banco acessível. **Ativação:** `POST /audit`.

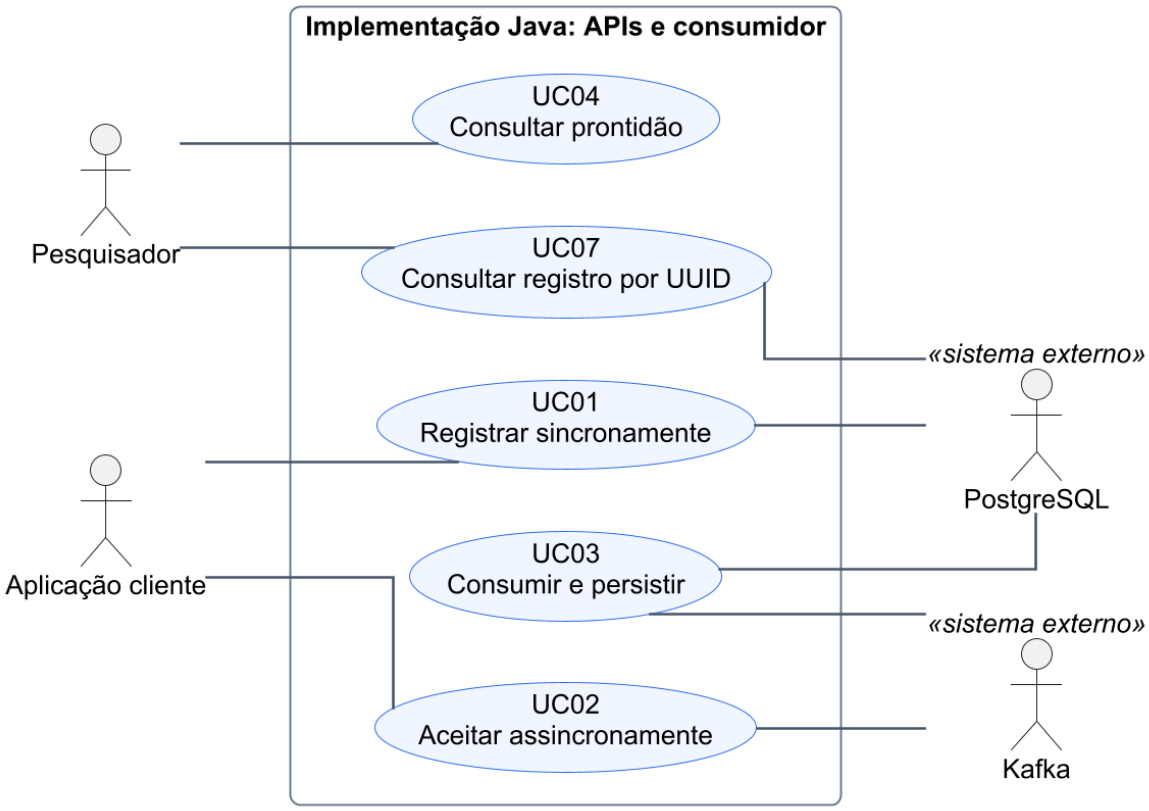


FIGURA 2 – Casos de uso da aplicação operacional

FONTE: Elaborado pelo autor (2026).

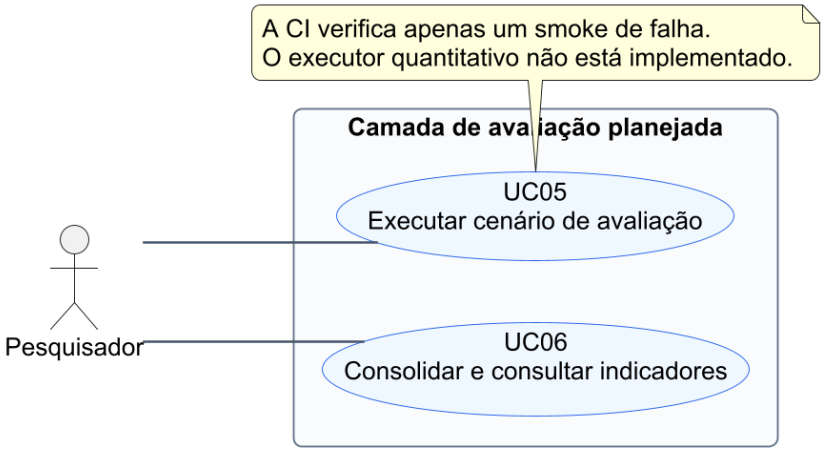


FIGURA 3 – Casos de uso da camada de avaliação

FONTE: Elaborado pelo autor (2026).

Pós-condição: linha inserida ou reentrega idêntica reconhecida. **Regras:** RN01–RN05 e RN08.

a) A aplicação envia o evento à API síncrona.

b) O contrato é validado e recebe os valores padrão permitidos.

c) O repositório calcula a representação determinística e o SHA-256.

d) Executa `INSERT ON CONFLICT DO NOTHING`, verificando o resultado.

e) A transação é concluída; o marco pós-commit é registrado.

f) A API retorna 201, UUID, hash, status e timestamp de inserção.

A1 — Reentrega: quando o UUID já existe, uma nova consulta na mesma transação em `READ COMMITTED` obtém a linha. Hash igual produz `duplicate`; nenhuma nova linha é criada. **E1 — Contrato inválido:** retorna 422 antes de chamar o repositório. **E2 — Conflito:** hash diferente provoca rollback e resposta 409 com `event_id_content_conflict`, preservando a linha original. **E3 — Banco indisponível:** retorna 503 com `database_unavailable`. Uma falha durante o commit pode deixar o resultado incerto; consulta e reenvio devem preservar o evento completo.

4.3.1.2 UC02 — Aceitar auditoria assincronamente

Objetivo e atores: confirmar publicação sem aguardar persistência; aplicação cliente e Kafka. **Pré-condição:** API e publicador inicializados. **Ativação:** `POST /audit` na API assíncrona. **Pós-condição:** publicação confirmada individualmente pelo broker. **Regras:** RN01, RN02, RN06 e RN09.

a) A API valida o mesmo contrato da variante síncrona.

b) Serializa os campos completos e usa `entity_id` como chave.

c) O publicador chama `send(...).get(...)` e aguarda a confirmação individual dentro do prazo.

d) Registra o ACK, incluindo tópico, partição e offset, e retorna 202.

A1 — Reenvio: a API não consulta duplicidade; o consumidor e o banco reconhecem eventos idênticos. **E1 — Contrato inválido:** retorna 422. **E2 — Buffer do produtor cheio:** retorna 503 com `acceptance=rejected`. **E3 — Timeout ou erro de confirmação:** retorna 503 com `acceptance=unknown`. A mensagem pode ter sido entregue; a resposta não autoriza assumir perda. **Limite:** ACK de publicação e persistência são pós-condições distintas. O consumidor pode processar a mensagem antes ou depois de o cliente receber a resposta.

4.3.1.3 UC03 — Consumir e persistir evento

Objetivo e atores: processar ou reter uma falha identificável; Kafka e PostgreSQL. **Pré-condições:** consumidor inscrito no tópico e configuração de retry válida. **Ativação:** mensagem retornada por `poll`. **Pós-condição:** persistência idempotente ou retenção confirmada na DLQ, seguida da confirmação da posição de origem. **Regras:** RN03–RN05, RN07, RN09 e RN11.

a) O consumidor recebe a mensagem e recupera o identificador da execução.

b) Valida JSON, contrato e presença explícita de UUID e instante.

c) Registra a tentativa e chama o repositório comum.

d) Depois da transação ou reconhecimento de duplicata, registra o resultado.

e) Confirma o offset de origem de forma síncrona e retorna à espera.

A1 — Reentrega idêntica: preserva a linha e confirma a posição após reconhecimento de duplicata. **A2 — Falha transitória:** repete a persistência até o limite configurado, com backoff exponencial limitado e registro das tentativas. **E1 — Mensagem inválida, conflito ou tentativas esgotadas:** publica envelope na DLQ com identidade da origem, motivo e conteúdo original, aguarda ACK e então confirma a origem. **E2 — DLQ não confirmada:** retoma a sessão sem confirmar a mensagem de origem. Mensagens posteriores não justificam avançar sua posição. **E3 — Falha de commit de offset:** retoma a sessão; a persistência ou a publicação em DLQ já pode ter ocorrido. A reentrega é tratada por UUID no banco ou pela identidade da origem na conciliação da DLQ. **E4 — Dependência indisponível:** o laço externo aguarda e reinicia a sessão; a política de restart do Compose cobre encerramento do processo. Mensagens já retidas na DLQ exigem inspeção e não retornam automaticamente ao fluxo principal.

4.3.1.4 UC04 — Consultar prontidão

Ator: pesquisador. **Ativação:** `GET /health`. **Pré-condição:** processo HTTP iniciado. **Fluxo principal:** verificar acesso ao esquema PostgreSQL na API síncrona ou existência e liderança das partições do tópico na API assíncrona; retornar 200 com `ready`. **Alternativa:** `GET /live` retorna `alive` para verificar apenas a resposta do processo. **Exceção:** dependência sem prontidão produz 503; processo parado causa falha de conexão. **Pós-condição:** resultado de uma verificação pontual, sem garantia de disponibilidade futura ou de conclusão de todos os eventos. Relaciona-se a RF10.

4.3.1.5 UC05 — Executar cenário de avaliação (planejado)

Ator: pesquisador. **Pré-condições:** ambiente isolado preparado, dados sintéticos e configuração identificada; protocolo congelado para coleta definitiva. **Ativação**

prevista: comando do executor e seleção do perfil. **Fluxo previsto:** criar diretório exclusivo; registrar manifesto; preparar eventos; executar aquecimento e carga; coletar logs, recursos e offsets; interromper o PostgreSQL em C4; registrar a retomada; aguardar drenagem; consolidar arquivos. **Alternativas:** teste dos instrumentos ou execução conforme protocolo definitivo. **Exceção prevista:** configuração incompleta ou falha de coleta deve ser registrada; validação funcional não vira resultado definitivo. **Pós-condição esperada:** artefatos verificáveis da execução, com indicação de completude. O projeto Java atual contém apenas *smokes* de CI, sem esse executor. Relaciona-se a RF12, RF13 e RN10.

4.3.1.6 UC06 — Consolidar e consultar indicadores (planejado)

Ator: pesquisador. **Pré-condição:** arquivos brutos de execução disponíveis. **Ativação prevista:** execução de um normalizador ainda não adaptado ao Java. **Fluxo previsto:** verificar arquivos; correlacionar UUIDs; separar fases; calcular métricas e unidades; registrar inconsistências; produzir tabelas e relatório. **Alternativa:** inspecionar lag, reentregas, retry, DLQ e recuperação. **Exceção:** ausência de instrumentação é sinalizada e não substituída por medição zero. **Pós-condição esperada:** indicadores rastreáveis aos dados brutos. A interface dedicada dos wireframes permanece proposta. Relaciona-se a RF11, RF14 e RN10.

4.3.1.7 UC07 — Consultar registro por UUID

Ator: pesquisador ou aplicação cliente. **Pré-condições:** API síncrona e banco acessíveis. **Ativação:** `GET /audit/{event_id}`. **Fluxo principal:** validar UUID, consultar a projeção da linha e retornar 200 com hash e timestamp. **Alternativa:** após reenvio, conferir a preservação do hash. **Exceções:** UUID inválido retorna 422; linha não localizada, 404; banco indisponível, 503. **Pós-condição:** existência ou ausência observada naquele instante; 404 durante processamento assíncrono não prova perda. Relaciona-se a RF17 e RN04.

4.3.2 Diagramas de atividades

A Figura 4 representa a inserção síncrona e as decisões de duplicidade e conflito. As Figuras 5 e 6 separam o aceite da API produtora e o processamento do consumidor. Os dois processos assíncronos não dependem da ordem em que a resposta HTTP chega ao cliente.

A atividade de persistência com retry corresponde ao laço limitado de UC03. O caminho de quarentena inclui a espera pelo ACK da DLQ. Um término da iteração não significa encerramento do serviço: o laço externo mantém a espera ou a reconexão, conforme indicado nas ações.

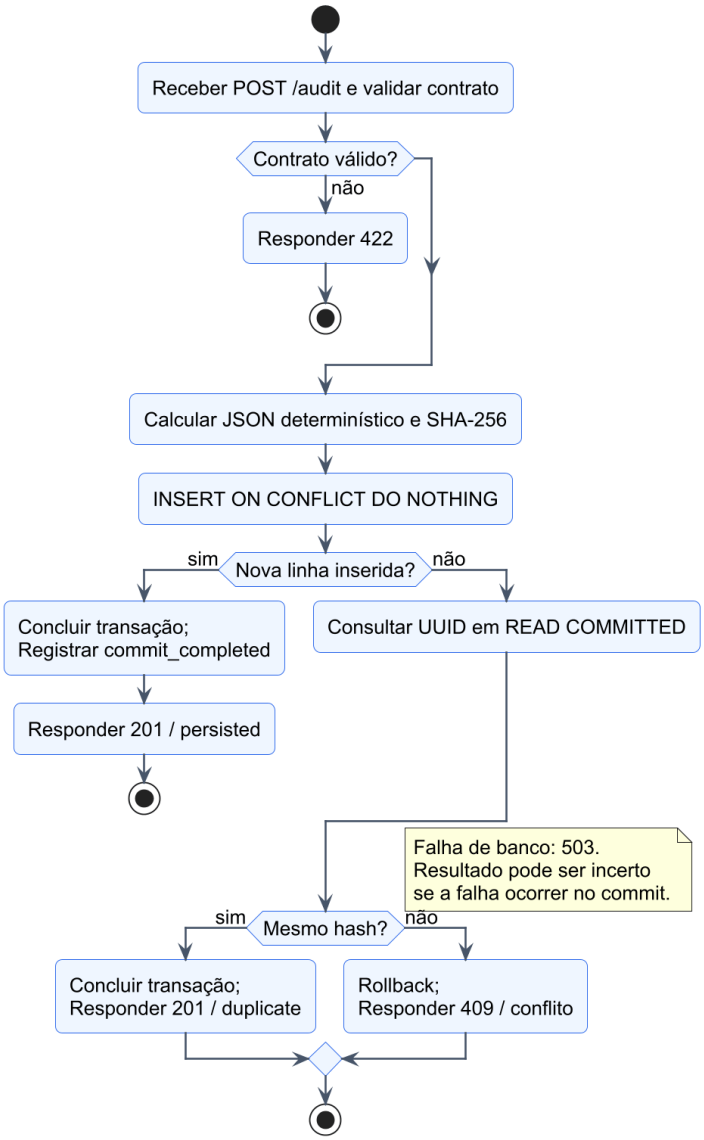


FIGURA 4 – Atividade síncrona: transação, duplicidade e conflito

FONTE: Elaborado pelo autor (2026).

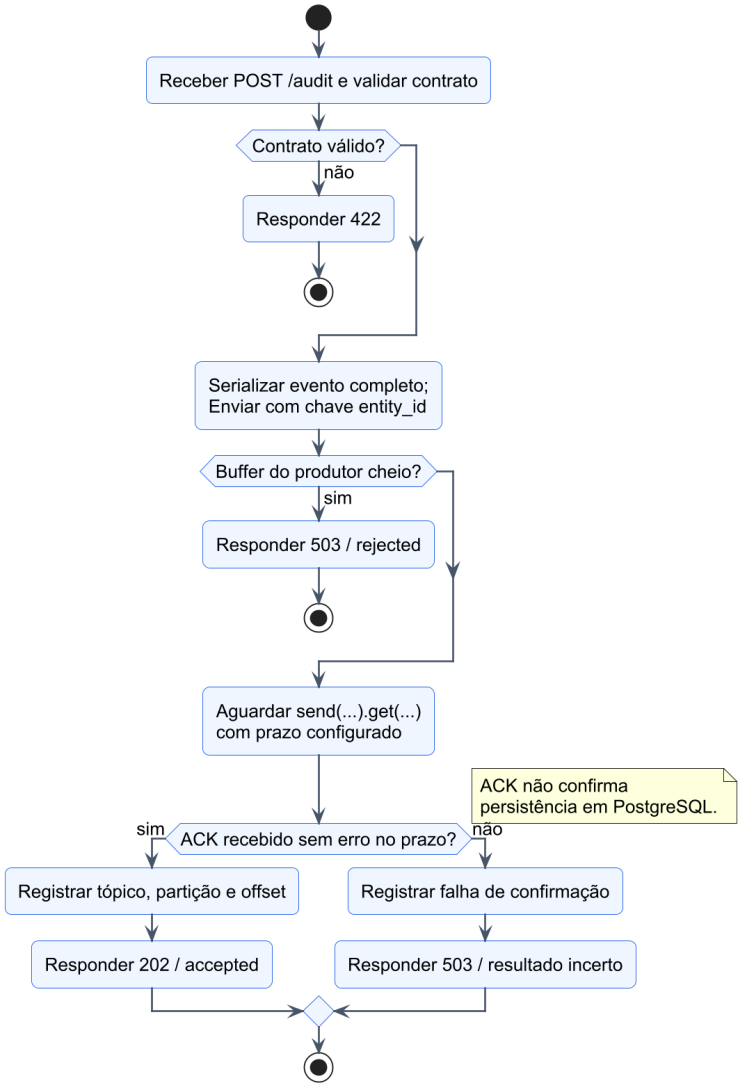


FIGURA 5 – Atividade da API produtora e confirmação de publicação

FONTE: Elaborado pelo autor (2026).

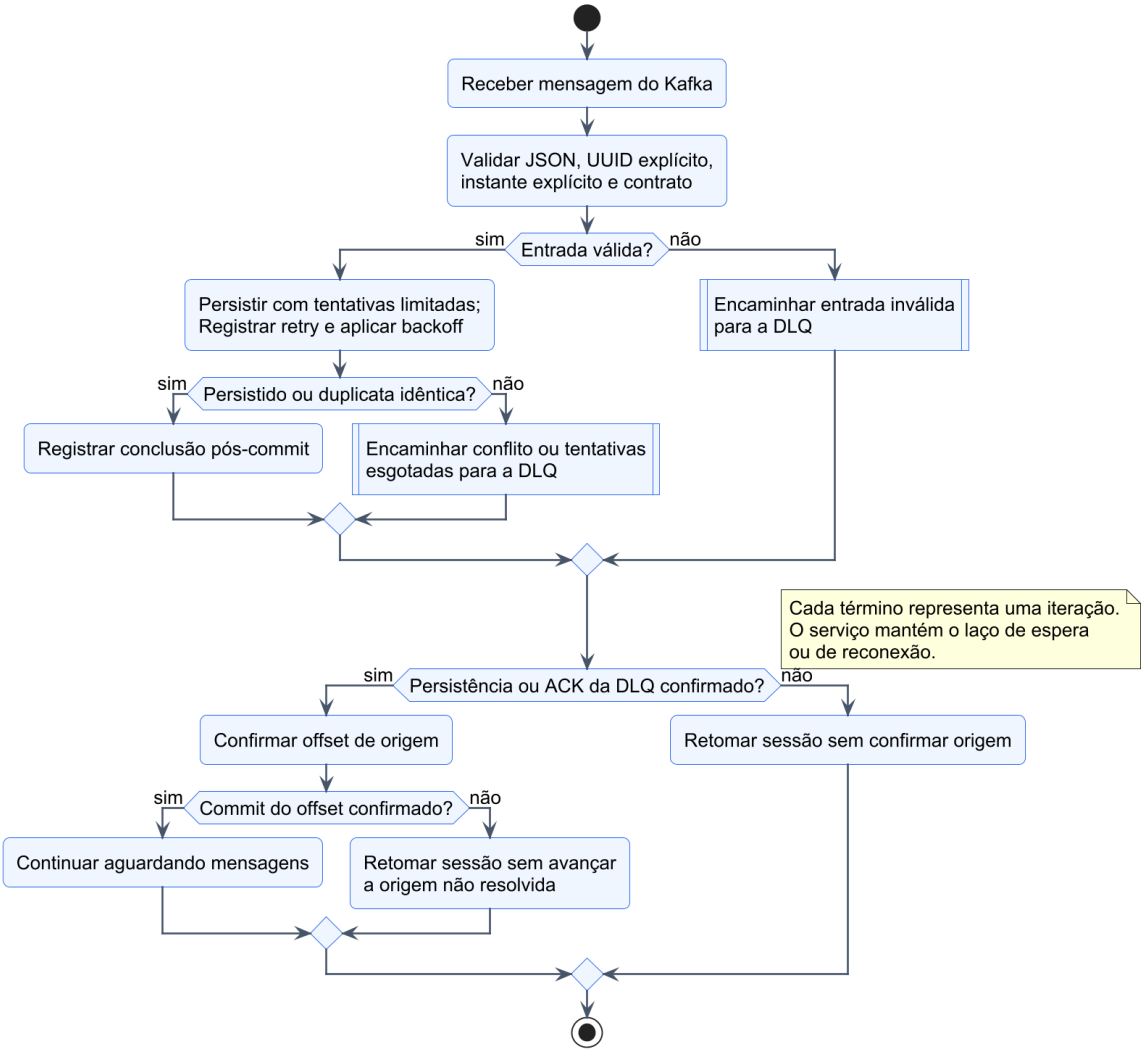


FIGURA 6 – Atividade do consumidor, retry e confirmação da origem

FONTE: Elaborado pelo autor (2026).

4.3.3 Diagramas de classes

A Figura 7 apresenta os *records* do contrato e o repositório JDBC. A Figura 8 mostra as classes de entrada e processamento dos três módulos executáveis, além das dependências compartilhadas. Nenhuma herança ORM é representada porque o projeto Java usa Spring JDBC e uma migração SQL.

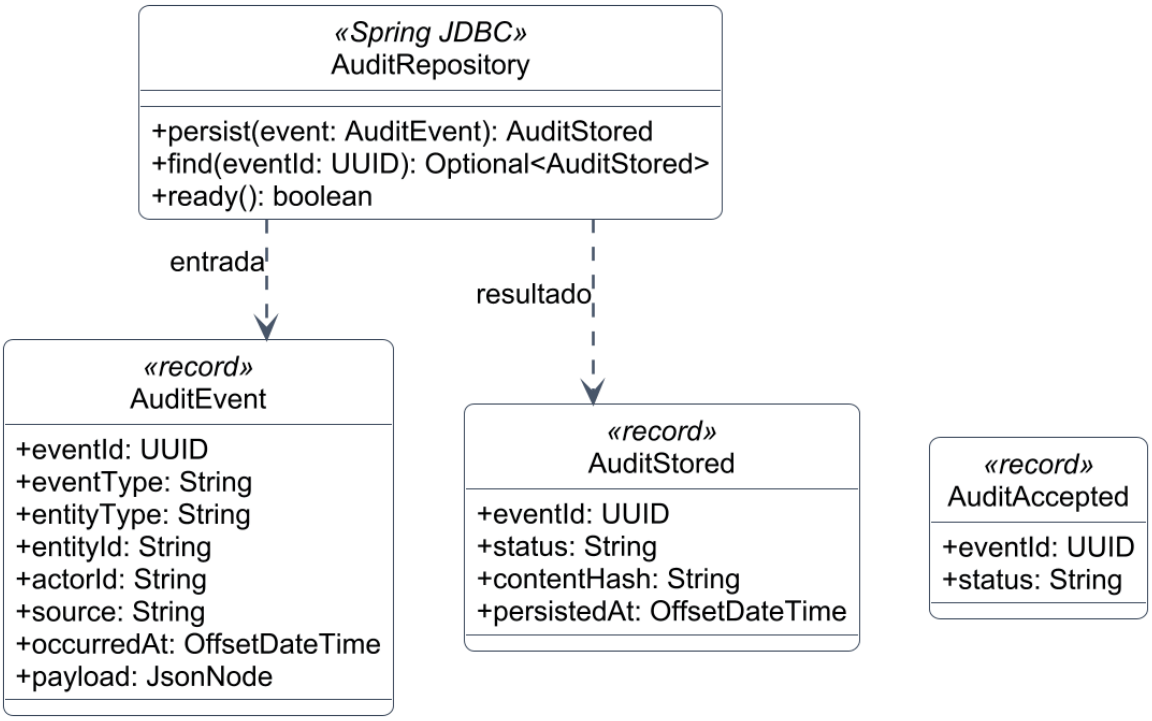


FIGURA 7 – Classes de contrato e persistência
FONTE: Elaborado pelo autor (2026).

A notação + indica membros públicos. A validação pertence ao contrato `AuditEvent`. A serialização e o hash ficam na `CanonicalEventCodec`; a transação, na `AuditRepository`; e os marcos, na `MilestoneLog`. O broker é infraestrutura, não classe de domínio. O `AuditMessageProcessor` chama `commitSync` sobre o consumidor Kafka depois da persistência ou do ACK da DLQ.

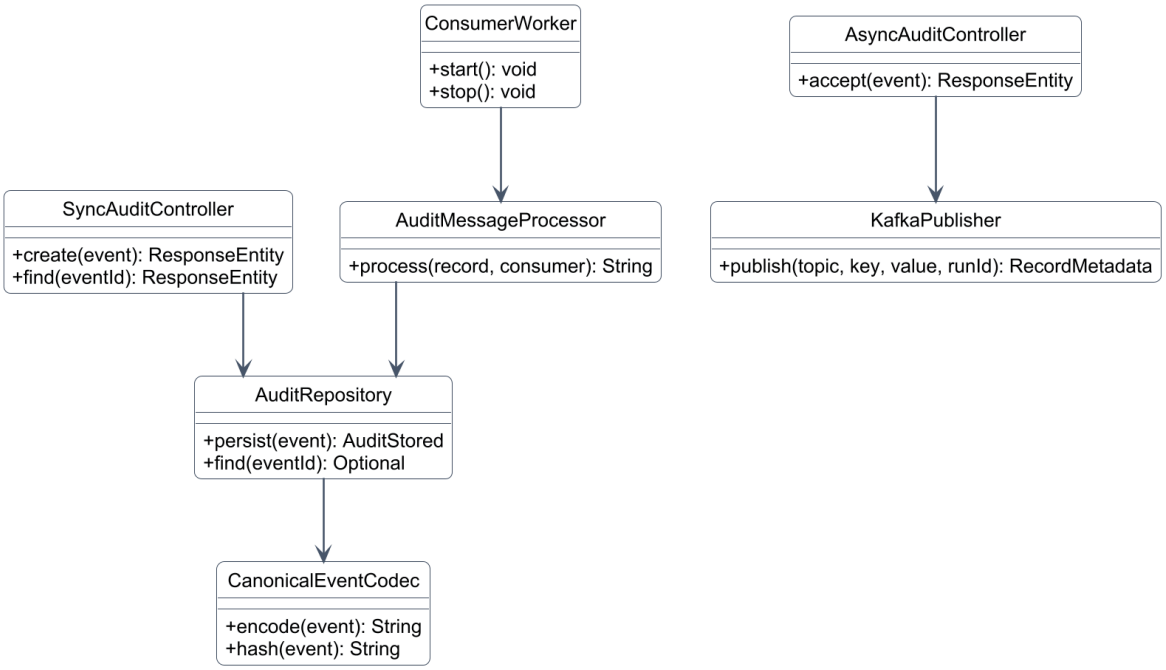


FIGURA 8 – Classes de aplicação e processamento Java
FONTE: Elaborado pelo autor (2026).

4.3.4 Diagramas complementares

As Figuras 9 e 10 detalham a ordem entre transação, resposta, ACK e confirmação de posição. A Figura 11 sintetiza as dependências entre os módulos Maven; o caminho de publicação na DLQ pelo consumidor é detalhado na atividade, na sequência e na arquitetura assíncronas para manter esta vista legível. As Figuras 12 e 13 distinguem os fluxos arquiteturais e a participação da DLQ.

4.3.4.1 Diagrama de implantação

A Figura 14 localiza os cinco serviços permanentes na rede Docker Compose e separa o cliente de carga, executado pelo pesquisador, dos processos de aplicação. Os serviços `kafka-init` e `kafka-volume-init` são transitórios e, por isso, aparecem na anotação do diagrama, não como processos permanentes. As portas publicadas estão restritas ao *loopback* do host. Os volumes nomeados conservam os dados do PostgreSQL e os logs do Kafka entre reinícios de contêiner. Esta é uma implantação experimental com um broker, uma partição por tópico e um banco; não representa alta disponibilidade.

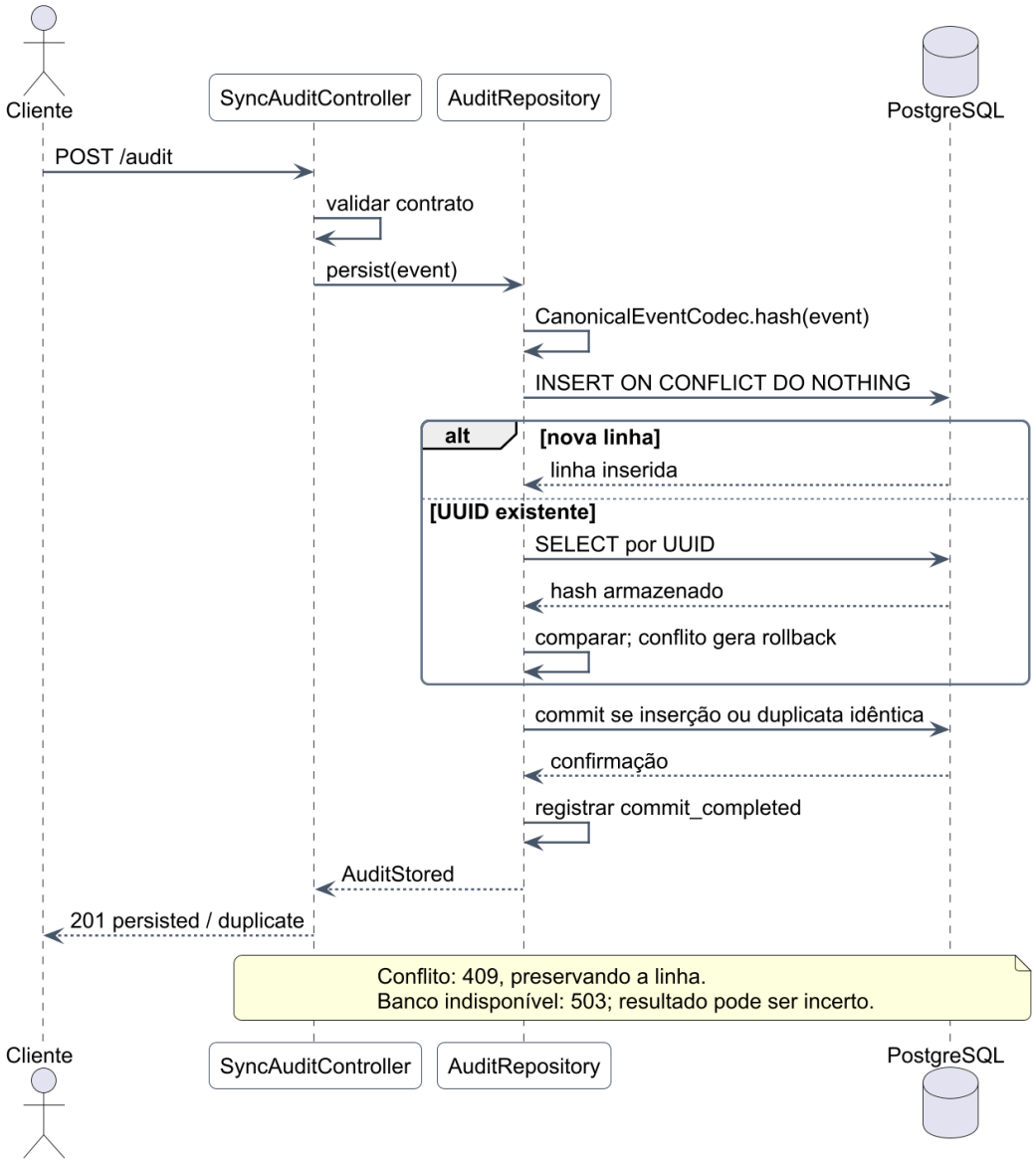


FIGURA 9 – Sequência síncrona com inserção idempotente

FONTE: Elaborado pelo autor (2026).

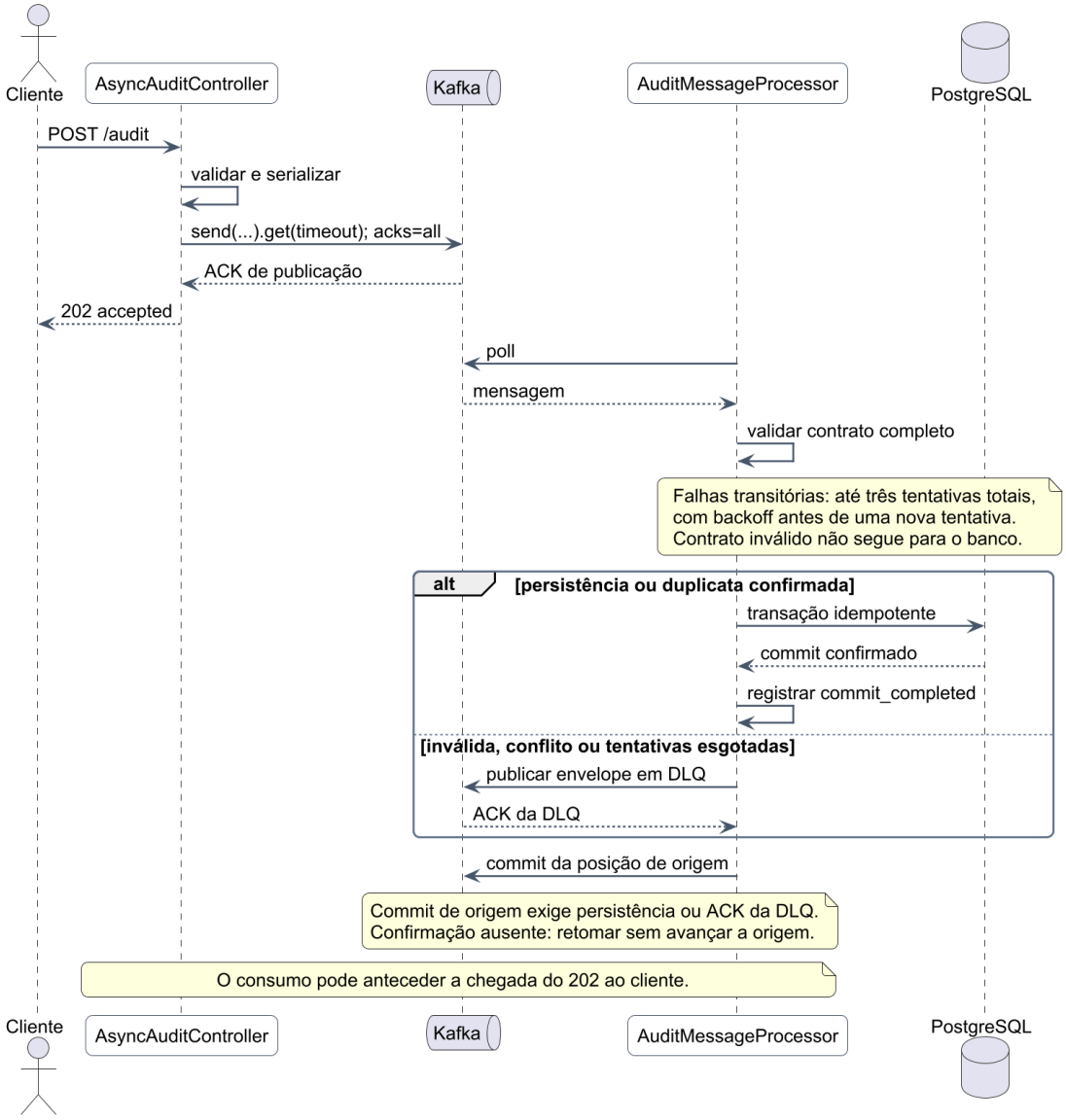


FIGURA 10 – Sequência assíncrona com ACK e tratamento de falhas

FONTE: Elaborado pelo autor (2026).

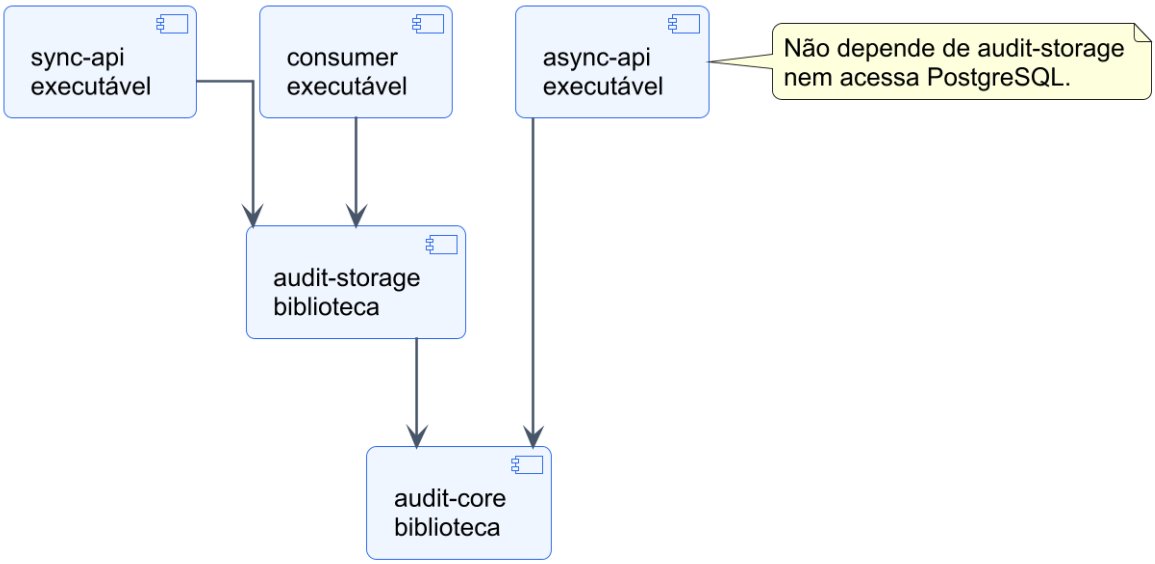


FIGURA 11 – Dependências entre os cinco módulos Maven

FONTE: Elaborado pelo autor (2026).

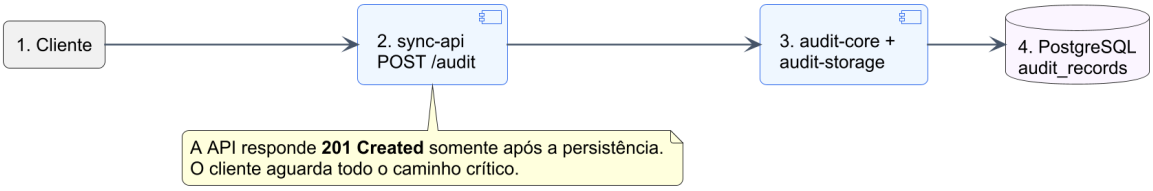


FIGURA 12 – Arquitetura da variante síncrona

FONTE: Elaborado pelo autor (2026).

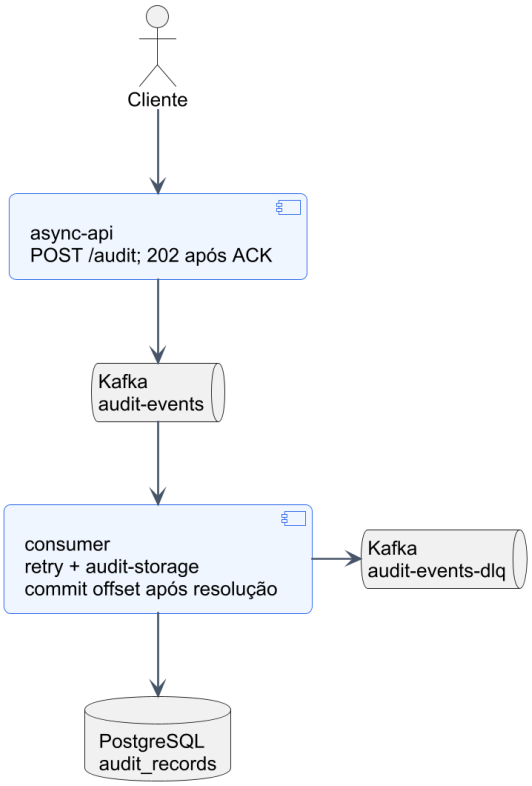


FIGURA 13 – Arquitetura da variante assíncrona e DLQ

FONTE: Elaborado pelo autor (2026).

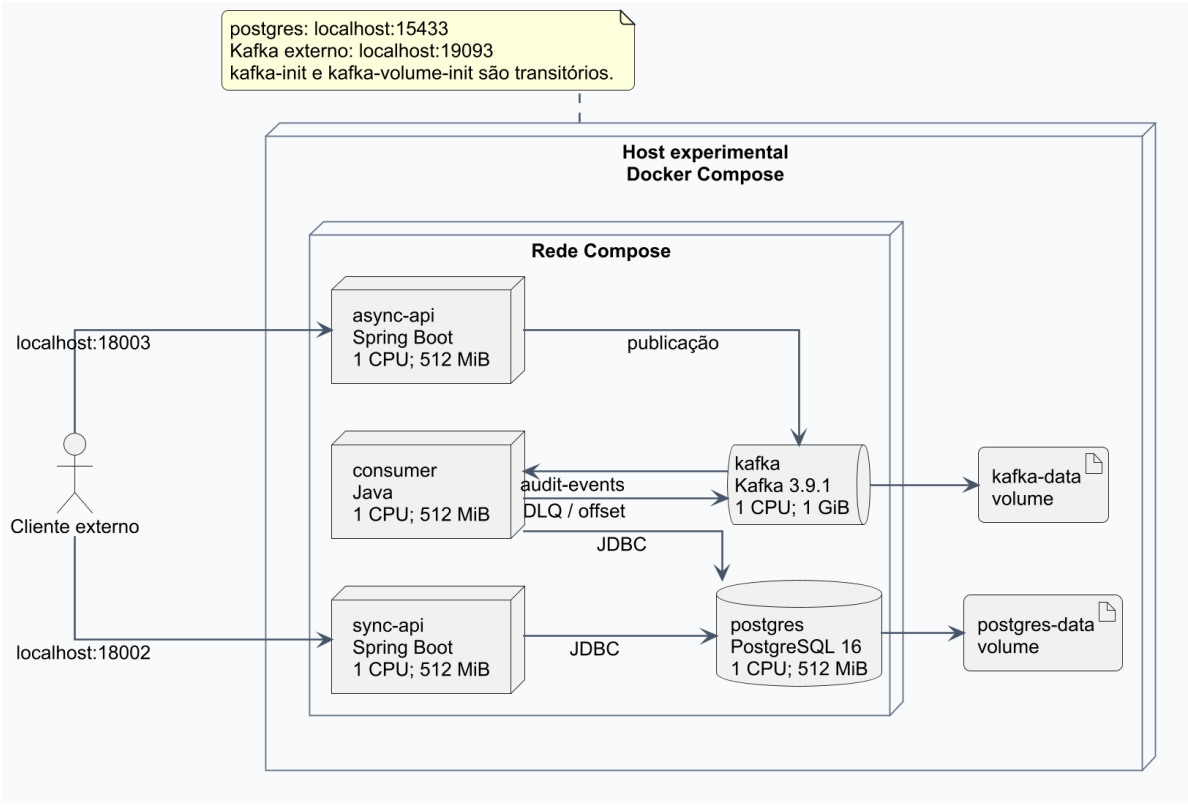


FIGURA 14 – Implantação real do protótipo no Docker Compose

FONTE: Elaborado pelo autor (2026).

4.3.5 Rastreabilidade entre requisitos, regras e UML

A Tabela 3 relaciona cada requisito funcional às regras e aos elementos que o representam. Os elementos planejados não indicam código já existente. A cobertura funcional verificada é delimitada na Seção 4.6.3.

TABELA 3 – Rastreabilidade funcional e comportamental

Requisitos	Regras	Casos e elementos de modelagem
RF01, RF02	RN01, RN02	UC01, UC02; AuditEvent e validação dos dois fluxos.
RF03, RF06, RF07	RN03, RN08	UC01; atividade e sequência síncronas; CanonicalEventCodec e AuditRepository.
RF04	RN06, RN09	UC02; KafkaPublisher; ACK na atividade e sequência assíncronas.
RF05, RF09, RF16	RN07, RN11	UC03; retry, DLQ e confirmação de origem; atividade do consumidor.
RF08	RN04, RN05	UC01 e UC03; decisões de hash e unicidade; tabela audit_records.
RF10	RN10	UC04; prontidão e vivacidade das APIs.
RF11, RF14	RN10	UC06 planejado; normalizador e wireframes de métricas/comparação.
RF12, RF13	RN07, RN10	UC05 planejado; protocolo e acompanhamento de backlog ainda incompletos.
RF15	RN01, RN02, RN06	UC01 e UC02; Swagger UI para envio manual e wireframe de formulário próprio ainda proposto.
RF17	RN04, RN05	UC07; consulta por UUID no repositório.

Fonte: Elaborado pelo autor (2026).

Síntese. Os casos de uso descrevem objetivos e exceções; atividades e sequências mostram as decisões; classes e componentes localizam suas responsabilidades. As condições da coleta definitiva permanecem ligadas ao protocolo metodológico.

4.4 MODELAGEM DE BANCO DE DADOS

A modelagem distingue o esquema operacional da extensão relacional proposta. O banco da implementação Java possui somente a tabela `audit_records`, criada pela migração Flyway `V1__create_audit_records.sql`. O acesso usa Spring JDBC; não há mapeamento ORM nem rotina de adaptação de linhas históricas da versão Python. Marcos de execução são emitidos nos logs da aplicação, fora do banco operacional. A coleta estruturada de métricas e tentativas para o experimento ainda será desenvolvida. As tabelas adicionais desta seção permanecem propostas opcionais.

4.4.1 Modelo conceitual (MER)

A Figura 15 apresenta o núcleo e a Figura 16 apresenta a extensão proposta. No núcleo atual, a entidade *Registro de auditoria* representa o fato persistido e contém identificação, origem, instante, entidade, ator, conteúdo e hash. Não há relacionamento entre tabelas no esquema existente. O hash é atributo do registro, pois há exatamente um digest armazenado por linha; criar uma entidade *EventHash* independente não acrescentaria informação necessária ao recorte e criaria uma sincronização adicional.

A extensão proposta introduz *Log de processamento*, *Tentativa de recuperação* e *Amostra de métrica*. Um UUID pode aparecer em zero ou muitos logs, inclusive quando nenhuma linha de auditoria foi persistida. Cada log representa uma tentativa ou etapa observada. Um log pode originar zero ou muitas tentativas de recuperação; cada tentativa referencia um log. Métricas agregadas são associadas ao identificador da execução, e não obrigatoriamente a um único evento.

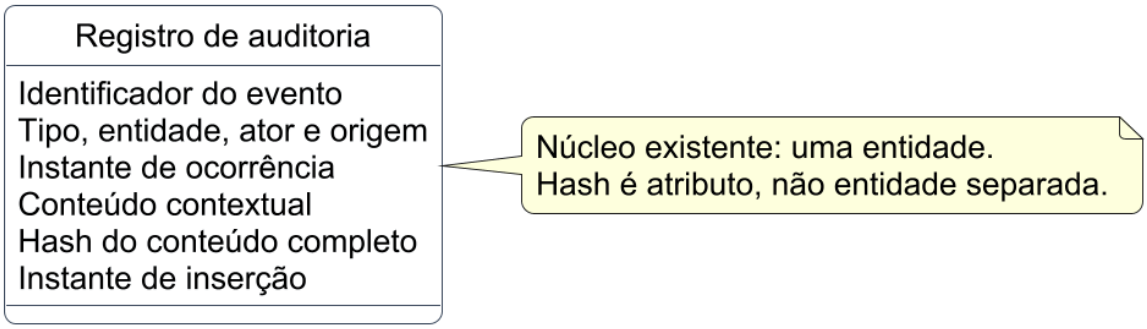


FIGURA 15 – Modelo conceitual do núcleo existente
FONTE: Elaborado pelo autor (2026).

A associação entre registro e log é de correlação por UUID, não de existência obrigatória: um erro pode ocorrer antes da primeira inserção. Não se estabelece uma chave estrangeira obrigatória de log para `audit_records`, pois isso impediria registrar justamente a falha de persistência. A tentativa de recuperação, por sua vez, referencia um log já existente. Essa distinção preserva a capacidade de registrar eventos rejeitados, falhas e reentregas sem pressupor sucesso.

4.4.2 Modelo lógico

No esquema atual, `audit_records.event_id` é chave primária; `entity_id` possui o índice secundário `idx_audit_records_entity_id`; os atributos do evento e o hash são não nulos. A coluna `persisted_at` é anulável no DDL, embora novas inserções recebam `clock_timestamp()` por padrão. A Figura 17 representa somente a tabela implementada. Não existem chaves estrangeiras ou relacionamentos entre tabelas. A restrição de unicidade protege contra inserções concorrentes com o mesmo UUID. O hash é calculado na aplicação e armazenado como texto; não há restrição SQL de comprimento do digest.

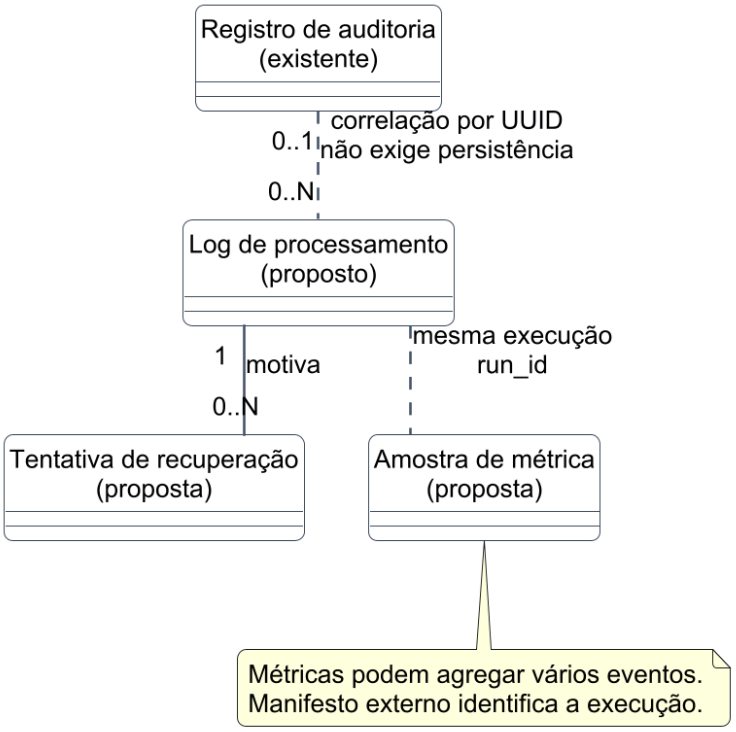


FIGURA 16 – Modelo conceitual da extensão de observabilidade proposta

FONTE: Elaborado pelo autor (2026).

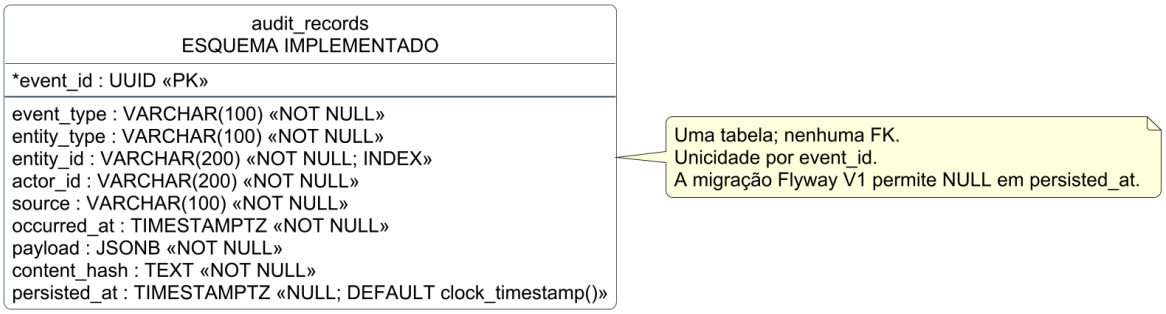


FIGURA 17 – Modelo lógico do banco implementado

FONTE: Elaborado pelo autor (2026).

4.4.3 Modelo físico implementado

O arquivo `audit-storage/src/main/resources/db/migration/V1__create_audit_records.sql` cria a tabela no PostgreSQL. Sua chave primária sobre `event_id` é a única restrição de unicidade explícita. `entity_id` recebe índice não único `idx_audit_records_entity_id`. Os nove campos obrigatórios são `event_id`, `event_type`, `entity_type`, `entity_id`, `actor_id`, `source`, `occurred_at`, `payload` e `content_hash`. O décimo campo, `persisted_at`, é anulável e possui `DEFAULT clock_timestamp()`. Não existe chave estrangeira, gatilho, exclusão lógica ou tabela de usuários nesse esquema.

O Flyway aplica a migração V1 na inicialização da API síncrona e do consumidor sobre o banco novo desta versão. Não há migração automática do banco Python nem compatibilidade retroativa demonstrada. A garantia de idempotência combina a PK, o `INSERT ON CONFLICT DO NOTHING` e a comparação do `content_hash` no `AuditRepository` após a tentativa de inserção. Portanto, o modelo físico deve ser entendido junto à regra implementada na aplicação, e não como uma restrição SQL de igualdade do conteúdo.

4.4.4 Extensão lógica opcional

A Figura 18 apresenta, separadamente, a possível extensão de observabilidade. As entidades destacadas como *propostas* não foram criadas pelo código e não integram o banco utilizado nos testes.

Na extensão, `processing_log.id`, `retry_log.id` e `metrics.id` são chaves primárias propostas. `retry_log.processing_log_id` referencia o log que motivou a tentativa. `run_id` correlaciona os três conjuntos com o manifesto externo de execução. Não se cria uma tabela de execuções nesta proposta inicial; caso isso seja feito posteriormente, o manifesto poderá ser normalizado em uma entidade própria.

A unicidade proposta de `(run_id, event_id, component, attempt)` em logs identifica uma tentativa por componente. Índices por execução, UUID e instante apoiam conciliação. Status e unidades deverão ser validados pela camada de coleta. Migrações e consultas dessas tabelas são atividades futuras, não executadas para produzir este texto. A instrumentação deverá ser equivalente entre variantes e ter seu custo considerado na análise experimental.

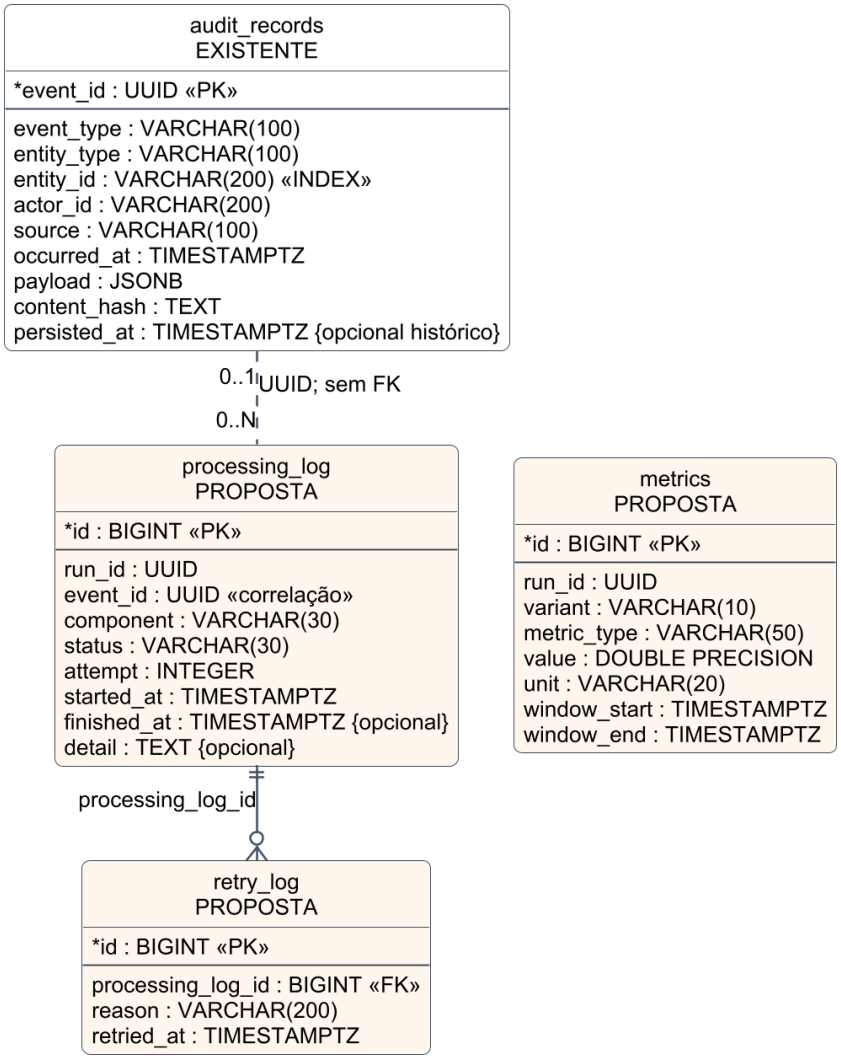


FIGURA 18 – Modelo lógico da extensão proposta, separado do esquema atual
FONTE: Elaborado pelo autor (2026).

4.4.5 Dicionário de dados

4.4.5.1 Tabela audit_records — existente

A Tabela 4 descreve o esquema efetivo. Os campos do evento e o hash são não nulos; `persisted_at` admite nulo pelo DDL da migração Java. No recebimento HTTP, `event_id`, `occurred_at` e `payload` podem ser omitidos e recebem valores padrão.

TABELA 4 – Dicionário do registro persistido

Campo	Tipo e tamanho	Obrigatoriedade, significado e validação
`event_id`	UUID, 128 bits	PK; não nulo. UUID informado deve ser válido; uuid4 gerado se ausente. Deve ser preservado em reenvios experimentais.
`event_type`	VARCHAR(100)	Não nulo; obrigatório na API, 1–100 caracteres. Tipo do fato observado; sem enumeração fechada implementada.

Campo	Tipo e tamanho	Obrigatoriedade, significado e validação
`entity_type`	VARCHAR(100)	Não nulo; obrigatório, 1–100 caracteres. Tipo da entidade referida.
`entity_id`	VARCHAR(200)	Não nulo; obrigatório, 1–200 caracteres; índice. Identificador da entidade e chave enviada ao Kafka.
`actor_id`	VARCHAR(200)	Não nulo; obrigatório, 1–200 caracteres. Ator sintético relacionado ao fato.
`source`	VARCHAR(100)	Não nulo; obrigatório, 1–100 caracteres. Sistema ou componente de origem.
`occurred_at`	TIMESTAMP WITH TIME ZONE	Não nulo. Instante de ocorrência; padrão UTC atual se omitido. Quando informado, o Java interpreta data ISO-8601 com offset. Não representa instante de commit.
`payload`	JSONB	Não nulo; objeto com padrão vazio. Sem limite de tamanho ou profundidade definido no DTO atual. Conteúdo contextual sintético.
`content_hash`	TEXT	Não nulo. SHA-256 calculado, 64 caracteres hexadecimais; o tipo SQL não impõe esse comprimento. Inclui os campos completos do evento.
`persisted_at`	TIMESTAMP WITH TIME ZONE	Anulável pelo DDL. Novas linhas recebem clock_timestamp() durante INSERT. Não representa conclusão do commit.

Fonte: Elaborado pelo autor (2026).

4.4.5.2 Tabela processing_log — proposta

A Tabela 5 descreve a proposta de uma tentativa por componente. Ela não substitui o evento original nem modifica sua identidade.

TABELA 5 – Dicionário proposto dos logs de processamento

Campo	Tipo e tamanho	Obrigatoriedade e regra proposta
`id`	BIGINT identity	PK, obrigatório. Identifica o log.
`run_id`	UUID	Obrigatório. Correlaciona com o manifesto de execução.
`event_id`	UUID	Obrigatório para eventos identificáveis. Correlação, sem FK para o registro persistido. Erros sem UUID ficam no log técnico do coletor.

Campo	Tipo e tamanho	Obrigatoriedade e regra proposta
`component`	VARCHAR(30)	Obrigatório. Identifica API síncrona, API produtora ou consumidor.
`status`	VARCHAR(30)	Obrigatório. Vocabulário proposto: accepted, persisted, duplicate, rejected, failed ou conflict.
`attempt`	INTEGER	Obrigatório; maior ou igual a 1. Número da tentativa por evento e componente na execução.
`started_at`	TIMESTAMPTZ	Obrigatório. Início da tentativa em UTC.
`finished_at`	TIMESTAMPTZ	Opcional enquanto a tentativa estiver aberta; ao concluir, não anterior ao início.
`detail`	TEXT	Opcional. Diagnóstico sem credenciais ou conteúdo sensível.

Fonte: Elaborado pelo autor (2026).

4.4.5.3 Tabela retry_log — proposta

A Tabela 6 descreve os campos da extensão para recuperação.

TABELA 6 – Dicionário proposto das tentativas de recuperação

Campo	Tipo e tamanho	Obrigatoriedade e regra proposta
`id`	BIGINT identity	PK, obrigatório.
`processing_log_id`	BIGINT	FK obrigatória para processing_log.id; identifica a tentativa anterior relacionada.
`reason`	VARCHAR(200)	Obrigatório. Motivo da retomada, sem dados sensíveis.
`retried_at`	TIMESTAMPTZ	Obrigatório. Instante da nova tentativa; não apenas um agendamento futuro.

Fonte: Elaborado pelo autor (2026).

O vínculo proposto ao log fornece execução, UUID e componente sem replicar esses campos. A política automática de retry está implementada no consumidor e emite marcos JSON no fluxo de logs; sua existência não depende da criação desta tabela.

4.4.5.4 Tabela metrics — proposta

A Tabela 7 descreve uma possível persistência relacional das métricas a serem coletadas no experimento Java.

TABELA 7 – Dicionário proposto das amostras de métricas

Campo	Tipo e tamanho	Obrigatoriedade e regra proposta
`id`	BIGINT identity	PK, obrigatório.
`run_id`	UUID	Obrigatório. Correlaciona com o manifesto.
`variant`	VARCHAR(10)	Obrigatório: sync ou async.
`metric_type`	VARCHAR(50)	Obrigatório. Identifica latência, throughput, erro, recurso ou backlog.
`value`	DOUBLE PRECISION	Obrigatório; valor finito. Ausência de amostra não deve ser registrada como zero.
`unit`	VARCHAR(20)	Obrigatório. Unidade explícita, como ms, eventos/s, bytes ou percentual.
`window_start`	TIMESTAMPTZ	Obrigatório. Início da janela de medição.
`window_end`	TIMESTAMPTZ	Obrigatório; igual ou posterior ao início.

Fonte: Elaborado pelo autor (2026).

A medição de conclusão utiliza o marco `commit_completed` emitido após a transação. Nem `occurred_at` nem `persisted_at` substituem esse marco. Consultas periódicas oferecem uma observação posterior da linha, sujeita ao intervalo de consulta, à duração da requisição e ao escalonamento; portanto, o intervalo configurado sozinho não estabelece um limite rígido para todo o atraso de observação.

Síntese. O esquema existente garante unicidade e conserva conteúdo e hash. O modelo proposto amplia a rastreabilidade de falhas e métricas sem impor uma FK que impeça registrar tentativas fracassadas e sem apresentar essas tabelas como já implementadas.

4.5 PROTOTIPAÇÃO DA INTERFACE

O núcleo atual é uma API, sem frontend de aplicação próprio. O Swagger UI permite documentar e testar manualmente as duas variantes, mas não corresponde aos wireframes desta seção nem executa o protocolo experimental. Os wireframes representam interfaces propostas de operação e inspeção, não capturas de telas funcionais. As telas não contêm resultados medidos; usam estados vazios para indicar o que deverá ser apresentado após instrumentação.

4.5.1 Envio de registro

A Figura 19 apresenta WF01, que permite escolher a variante e informar todos os campos do contrato. O usuário poderá gerar UUID e instante antes do envio e reutilizá-los em uma reentrega. A ação de envio mostrará código HTTP, corpo da resposta e UUID. Na variante assíncrona, a interface deverá distinguir aceitação de conclusão e não apresentar

202 como comprovante de persistência. Os campos obrigatórios e seus limites seguem o dicionário da Seção 4.4. Relaciona-se a RF01, RF02 e RF15.

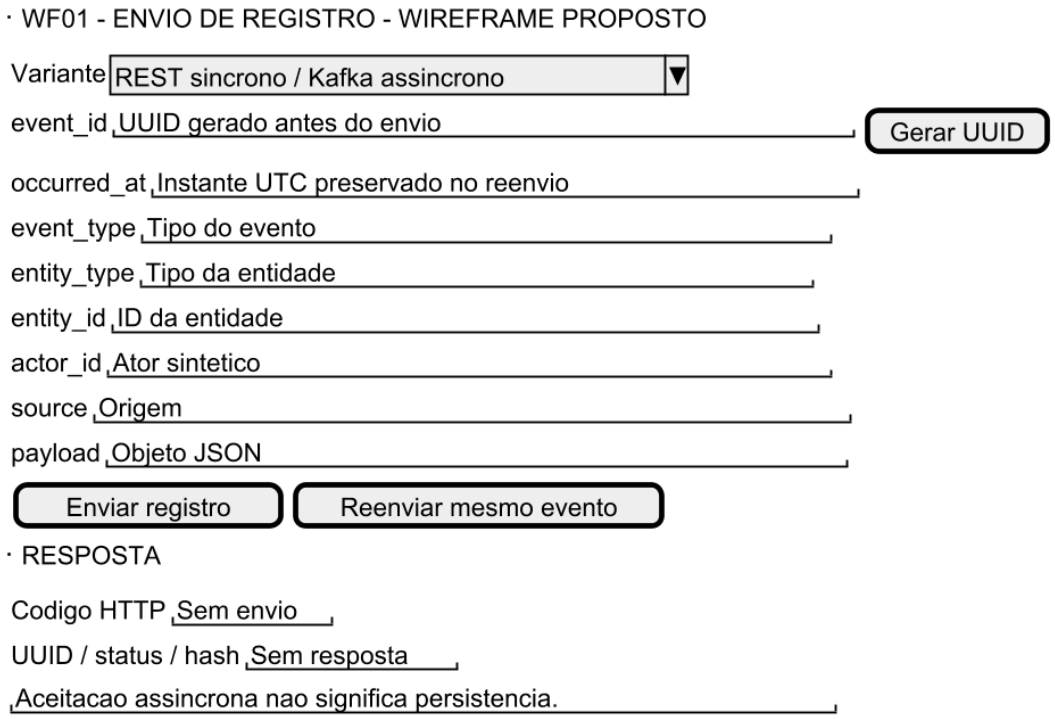


FIGURA 19 – WF01: wireframe proposto para envio de registros
FONTE: Elaborado pelo autor (2026).

4.5.2 Visualização de métricas

A Figura 20 apresenta WF02, que seleciona execução, variante e janela; apresenta taxa oferecida, throughput persistido, erros e percentis. Aceitação e conclusão ficam em campos distintos. O botão de exportação representa integração futura com os arquivos que serão produzidos pelo executor experimental Java. Falta de instrumentação deve aparecer como ausência de dados, não como medição igual a zero. Relaciona-se a RF11 e RF14.

4.5.3 Acompanhamento de backlog

A Figura 21 apresenta WF03, que permite inspecionar tópico, grupo e estado de coleta. Exibe offsets, diferença de posições e quantidade conciliada de eventos aceitos e persistidos. Esses números não são intercambiáveis: offsets dependem de partições e confirmações, enquanto a conciliação depende de UUIDs. O controle de falhas deverá exigir identificação do cenário e registrar os instantes de interrupção e retomada. Relaciona-se a RF12 e RF13.

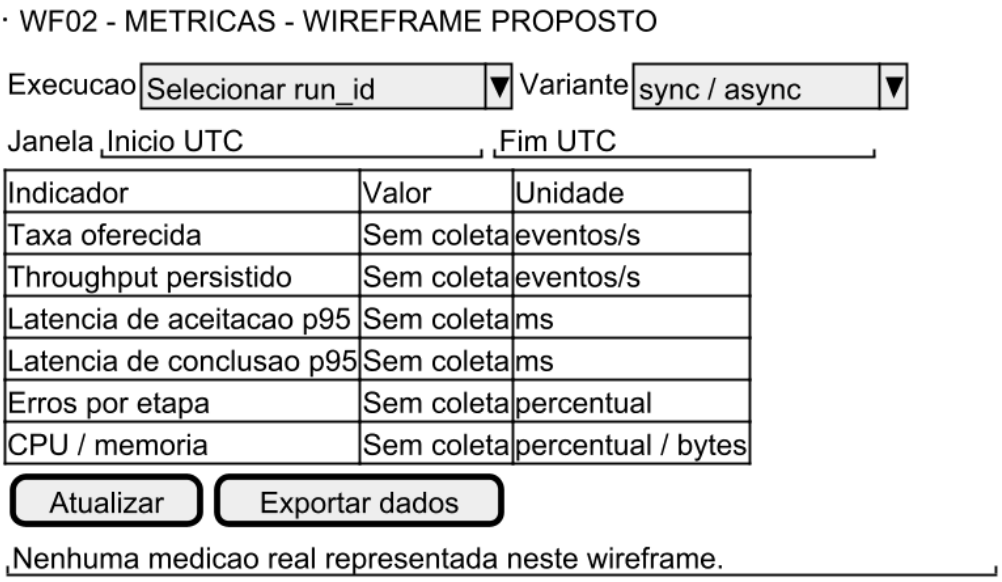


FIGURA 20 – WF02: wireframe proposto para métricas

FONTE: Elaborado pelo autor (2026).

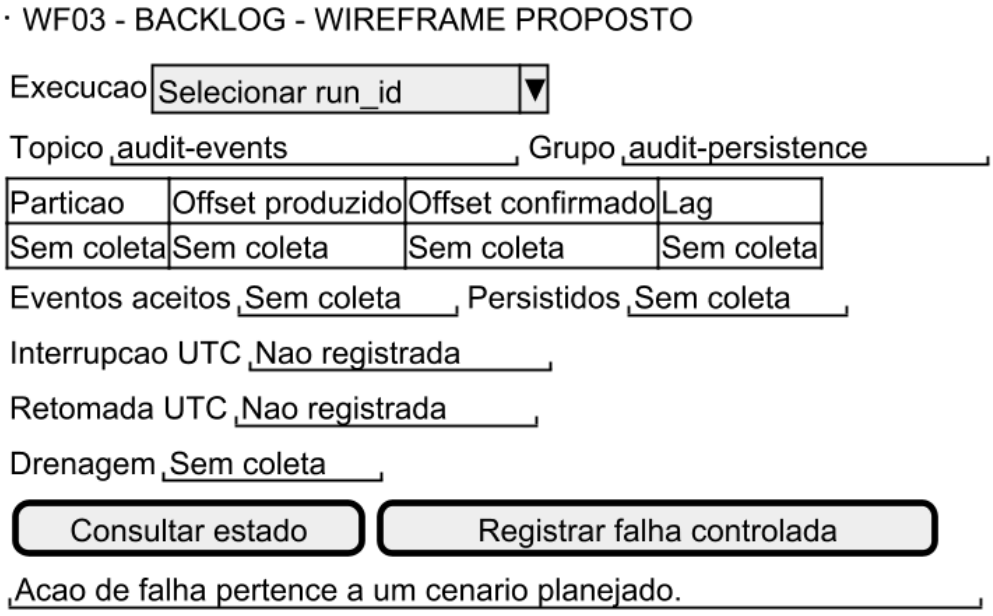


FIGURA 21 – WF03: wireframe proposto para backlog e recuperação

FONTE: Elaborado pelo autor (2026).

4.5.4 Comparação entre variantes

A Figura 22 apresenta WF04, que organiza lado a lado as mesmas métricas por variante. A interface deverá impedir comparação entre configurações incompatíveis ou apresentar explicitamente as diferenças. A latência de aceitação não substituirá a de conclusão. Não haverá rótulo de vencedor automático: a interpretação considera dispersão, recursos, falhas e limites de generalização. Relaciona-se a RF14 e RN10.

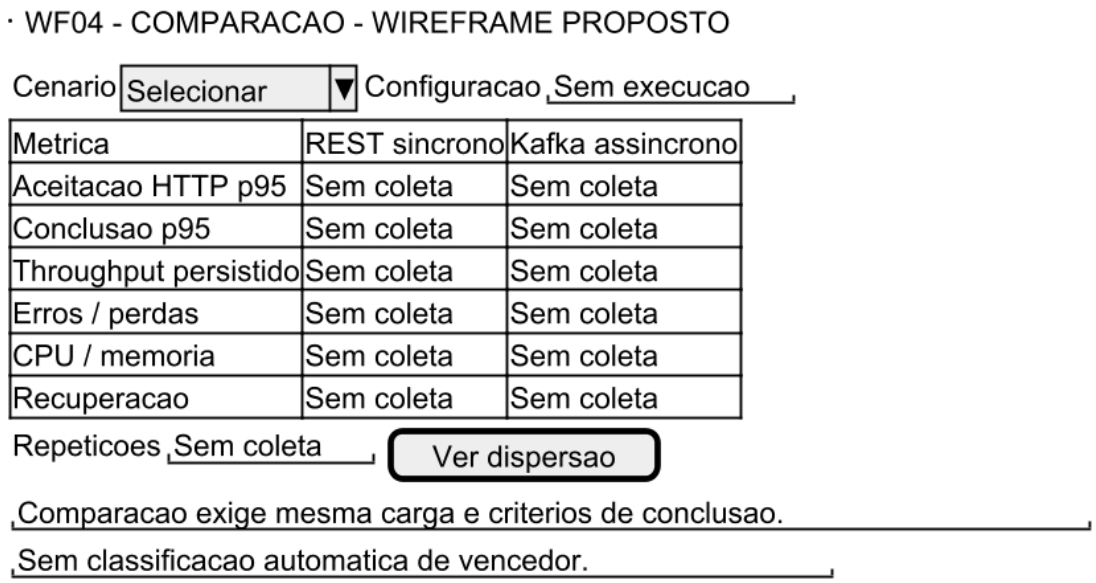


FIGURA 22 – WF04: wireframe proposto para comparação experimental
FONTE: Elaborado pelo autor (2026).

Síntese. Os quatro wireframes descrevem a interação desejada e os estados sem dados. A interface dedicada permanece proposta; os *smokes* e testes Java descritos na Seção 4.6.3 fornecem evidência funcional limitada, mas ainda não alimentam essas telas. As imagens desta seção representam exclusivamente prototipação de interface.

4.6 IMPLEMENTAÇÃO DO SISTEMA

Esta versão do protótipo foi implementada em Java e inspecionada em 29 de setembro de 2026. O projeto é uma implementação paralela do mesmo recorte experimental, não uma comparação entre linguagens. O código-fonte e sua configuração estão no repositório <https://github.com/ilann47/TCC-2026>, no diretório `implementacao-java`. As verificações funcionais descritas nesta seção não constituem a coleta quantitativa definitiva prevista no Capítulo 3.

4.6.1 Tecnologias e módulos

O projeto utiliza Java com bytecode alvo 21, Spring Boot 3.5.16, Maven Wrapper, driver PostgreSQL, Spring JDBC, Flyway, cliente Apache Kafka 3.9.1 e springdoc-openapi

2.9.1 para a documentação das APIs. A implantação descrita em `docker-compose.yml` utiliza PostgreSQL 16 e Kafka 3.9.1. O Maven compila cinco módulos: `audit-core`, `audit-storage`, `sync-api`, `async-api` e `consumer`. Os dois primeiros são bibliotecas compartilhadas; os três últimos são processos executáveis independentes.

TABELA 8 – Módulos Java e responsabilidades efetivas

Módulo	Responsabilidade
`audit-core`	Contrato `AuditEvent`, serialização determinística, SHA-256, publicação Kafka e marcos de observabilidade.
`audit-storage`	Migração Flyway V1, transação JDBC, consulta, comparação de hash e persistência idempotente.
`sync-api`	`POST /audit`, `GET /audit/{event_id}`, `/live` e `/health`; grava antes da resposta 201.
`async-api`	`POST /audit`, `/live` e `/health`; publica no Kafka e responde 202 depois do ACK. Não acessa o banco diretamente.
`consumer`	Consome o tópico, persiste, aplica retry, publica na DLQ quando necessário e confirma o offset depois da resolução.

FONTE: Elaborado pelo autor a partir do código Java (2026).

O Compose inicia os três processos executáveis, PostgreSQL e Kafka. A API síncrona expõe a porta local 18002 e a assíncrona, 18003; PostgreSQL e Kafka expõem 15433 e 19093 somente em *loopback*. O consumidor não expõe HTTP. Os serviços auxiliares `kafka-init` e `kafka-volume-init` preparam os tópicos e as permissões antes do uso. O ambiente possui um broker e uma partição por tópico; não representa alta disponibilidade. Os limites configurados são 1 CPU e 512 MiB para cada processo Java, 1 CPU e 512 MiB para PostgreSQL, e 1 CPU e 1 GiB para Kafka. Esses valores delimitam a configuração de desenvolvimento, sem constituir calibração científica.

A API síncrona hospeda uma única página Swagger UI na porta local 18002. Seu seletor carrega separadamente os contratos OpenAPI das variantes síncrona e assíncrona; a segunda API expõe o documento JSON, mas não outra interface Swagger. A configuração de CORS da API assíncrona permite à origem local da página consultar o contrato e executar requisições de teste. Essa ferramenta auxilia a inspeção manual das respostas 201 e 202, sem constituir frontend próprio, gerador de carga ou instrumento de medição do experimento.

4.6.2 Comportamento implementado

O contrato `AuditEvent` é compartilhado pelas duas APIs. Campos de texto obrigatórios respeitam os comprimentos definidos no DTO; `event_id`, `occurred_at` e `payload` recebem valores padrão apenas quando omitidos no HTTP. O consumidor exige UUID e instante explícitos na mensagem recebida para não regenerar identidade em uma reentrega. A serialização canônica e o SHA-256 são comuns às variantes.

A API síncrona chama `AuditRepository.persist`, que executa `INSERT ON CONFLICT DO NOTHING` em transação `READ COMMITTED`. Uma segunda submissão com

UUID e hash iguais preserva uma linha e retorna `duplicate`; hash diferente provoca conflito 409 e mantém o conteúdo original. O retorno 201 ocorre após o término da transação. `persisted_at` é o horário atribuído durante a inserção, enquanto o marco `commit_completed` é emitido depois do retorno da transação.

A API assíncrona usa `KafkaPublisher.publish`, que aguarda a conclusão de `send(...).get(...)` dentro de um prazo configurado. `acks=all` e a idempotência do produtor estão habilitados. O retorno 202 confirma a publicação no broker configurado, mas não confirma a gravação no PostgreSQL. Uma falha ou timeout de publicação retorna 503; em alguns casos a aceitação permanece incerta, exigindo conciliação pelo UUID antes de classificar o evento como perdido.

O consumidor usa commit manual de offsets. Após persistência ou reconhecimento idempotente, confirma a posição de origem. Mensagem inválida, conflito ou esgotamento das tentativas de banco seguem para `audit-events-dlq`; o offset de origem só é confirmado depois do ACK da DLQ. Se a publicação na DLQ ou o commit do offset falhar, a sessão é retomada sem confirmação segura da posição. A DLQ pode conter cópias repetidas após falha entre seu ACK e o commit de origem. Não existe reprocessador automático da DLQ: um evento aceito com 202 durante indisponibilidade prolongada do banco pode terminar em quarentena e não será persistido automaticamente quando o banco voltar.

4.6.3 Verificação funcional e limites da evidência

O Maven Wrapper executa build e testes unitários dos módulos Java. O fluxo de integração contínua publicado no repositório executou `mvnw verify`, iniciou PostgreSQL e Kafka em contêineres, verificou o caminho normal de registro síncrono e assíncrono e conferiu o seletor da documentação e a política CORS necessária ao teste manual. Um *smoke test* de C4 interrompeu o PostgreSQL e verificou a resposta 503 da API síncrona, a aceitação 202 da API assíncrona enquanto Kafka permanecia disponível e a retomada da saúde do banco. O registro de execução do fluxo está disponível em <https://github.com/ilann47/TCC-2026/actions/runs/36647710056>.

Essas verificações demonstram apenas os comportamentos observados no ambiente de CI. O *smoke test* C4 não prova que cada evento aceito durante a falha seja persistido depois da recuperação; tampouco mede latência, throughput ou tempo de drenagem de maneira comparativa. Em outubro de 2026, foram acrescentados scripts k6, um executor isolado, um analisador independente Java e um comparador de exportações do modelo comum. Esses instrumentos possuem testes próprios e preservam evidências locais. Sua validação funcional e o piloto são discutidos no Capítulo 5, sem substituírem a calibração, o congelamento dos parâmetros e as repetições da campanha definitiva.

4.6.4 Rastreabilidade da entrega

As fontes de verificação desta versão são o código Java versionado, a migração

SQL `V1__create_audit_records.sql`, os testes dos módulos, o arquivo Compose e o fluxo de integração contínua. Os diagramas desta entrega foram refeitos para corresponder a essas fontes; modelos de interface e tabelas adicionais continuam identificados como propostas. O repositório preserva o histórico de commits e a especificação técnica versionável em `implementacao-java/docs/especificacao`. Resultados experimentais serão apresentados somente após protocolo congelado, execução das repetições e conciliação dos dados brutos.

5 ANÁLISE DOS RESULTADOS

Este capítulo apresenta a campanha definitiva R2, concluída em 27 de outubro de 2026, e distingue seus resultados da validação funcional e da calibração. Foram observados os 65 UUIDs planejados: 64 coletas completas no critério original e uma parcial qualificada pela emenda A1, sem repetição ou substituição. A análise é descritiva, por execução e par de mesmo dataset, sem tratar os eventos como repetições independentes nem extrapolar a configuração local para REST ou Kafka em geral.

5.1 FERRAMENTAS DE CARGA E PILOTO

Foram examinadas as documentações oficiais do Apache JMeter e do Grafana k6. O Quadro 1 registra a comparação funcional utilizada na escolha do instrumento. Não foi executado um benchmark entre os geradores e, portanto, a escolha não afirma que k6 tenha melhor desempenho que JMeter.

QUADRO 1 – Comparação dos instrumentos de geração de carga

Critério	**Apache JMeter**	**Grafana k6**
Plano	Plano de teste JMX, interface gráfica e propriedades parametrizáveis.	Funções e cenários definidos em JavaScript.
Execução	Modo de linha de comando recomendado para carga; listeners gráficos aumentam o consumo.	Linha de comando e execução em contêiner.
Chegada aberta	Open Model Thread Group, classificado na documentação como experimental.	Executores de taxa constante e variável, independentes da duração das respostas.
Dados de saída	JTL/CSV; recomenda-se limitar os dados coletados.	JSON por linha, comprimível, com tags de correlação.

FONTE: Elaborado pelo autor com base nas documentações oficiais das ferramentas, consultadas em 6 out. 2026.

As orientações de execução em linha de comando e redução de listeners constam das boas práticas do JMeter[1]. A existência de um modelo aberto também no JMeter[2] evita atribuir essa capacidade exclusivamente ao k6. No k6, o executor de chegada constante inicia iterações segundo uma taxa programada, desde que existam usuários virtuais disponíveis[3]. A saída JSON permite preservar amostras e metadados por evento[4]. Essas fontes são documentação técnica dos instrumentos; não substituem os artigos científicos discutidos na revisão bibliográfica.

[1] <https://jmeter.apache.org/usermanual/best-practices.html>
[2] <https://jmeter.apache.org/usermanual/component_reference.html>, seção Open Model Thread Group.
[3] <https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/constant-arrival-rate/>
[4] <https://grafana.com/docs/k6/latest/results-output/real-time/json/>

O k6 foi escolhido pela integração com o protocolo já implementado: scripts versionados permitem gerar os UUIDs e os conteúdos sintéticos, separar aquecimento e medição, registrar a fase de cada amostra e enviar o identificador da execução no cabeçalho HTTP. A imagem utilizada é a versão 2.3.0, fixada por digest. O analisador Java cruza os dados do gerador com os marcos de transação e os registros exportados do PostgreSQL. A exportação por evento e a instrumentação também consomem recursos, razão pela qual precisam permanecer constantes nas comparações e ser reconhecidas como limitação.

5.1.1 Validação inicial dos instrumentos

Em 5 de outubro de 2026, foram realizadas duas execuções curtas válidas de conferência, à taxa de 2 eventos por segundo durante 10 segundos. A variante síncrona registrou 21 eventos e a assíncrona, 20. Em cada execução, todos os eventos observados foram associados a respostas esperadas, marcos de confirmação e registros com conteúdo e hash correspondentes. A diferença de uma iteração nos limites da execução reforça a necessidade de informar a carga efetiva, além da programada. Esses ensaios não usaram o mesmo conjunto entre as variantes e não estimam a carga de referência.

Cinco tentativas anteriores foram preservadas como inválidas por falha de conciliação dos logs do ambiente Docker, embora as linhas tivessem sido encontradas no banco. O problema foi investigado antes de aceitar novas coletas. Não se interpretou ausência de marco como perda de registro. O histórico de tentativas e seus UUIDs está preservado no relatório técnico do piloto e nas evidências locais.

O intervalo nativo de resposta HTTP do k6 é separado do intervalo instrumentado entre envio e retorno. A conclusão da persistência é observada pelo marco `commit_completed`, emitido após o retorno da transação. O campo `persisted_at`, preenchido no INSERT, não é utilizado como substituto desse marco. Uma resposta 202 confirma publicação no Kafka e não gravação no PostgreSQL.

5.1.2 Calibração e validação da revisão reduzida

A calibração de 26 de outubro de 2026 explorou taxas progressivas e confirmou 48 eventos por segundo em ambas as variantes, por 300 segundos medidos após aquecimento. Essa referência é local e não representa capacidade máxima. O relatório técnico preserva vinte coletas de calibração e validação, controles de instrumentação e identificação das tentativas rejeitadas; nenhuma constitui repetição definitiva.

O primeiro piloto reduzido, de 180 segundos, concluiu duas verificações funcionais de cinquenta registros. Na primeira referência assíncrona, foram enviados, aceitos e persistidos ao fim 8.641 registros, sem erros HTTP, iterações não iniciadas, DLQ ou ausência conciliada. Desses, 8.619 tiveram commit dentro da janela. Apesar da conciliação final completa, o pico de aceites ainda não gravados foi 21, acima do limite predeclarado de 12; o lag máximo amostrado foi 15 e o indicador de crescimento de cauda foi 0,01626

evento por segundo, acima do limite de 0,01. A coleta foi instrumentalmente íntegra, mas não elegível para liberar a referência naquele piloto. Ela foi preservada com identificador `bedbd896-c279-475a-94f1-712c88b7f904`.

A investigação identificou uma sonda administrativa Java periódica dentro da quota de CPU do broker, que também integra suas amostras de recursos. Essa constatação não demonstra causalidade exclusiva do pico. A revisão R2 preservou 48 eventos por segundo e os critérios, mas utilizou a sonda leve descrita no método e nove novos UUIDs de validação. Os nove casos foram aprovados e auditados em 27 de outubro de 2026, antes do congelamento do protocolo e de qualquer execução definitiva. Na referência de 180 segundos, cada variante enviou, aceitou e persistiu ao fim 8.641 eventos. A assíncrona apresentou pico de dez aceites sem commit, lag máximo amostrado de dois e inclinação da cauda de 0,0001115 evento por segundo, dentro dos limites predeclarados. O modelo comum manteve cinquenta linhas equivalentes nas duas variantes.

Nos três casos de falha do piloto R2 houve confirmação de recuperação operacional sustentada dentro da entrada ativa. A interrupção do PostgreSQL na assíncrona produziu dois eventos em DLQ e 2.159 persistidos, dentre 2.161 aceites; portanto, recuperação operacional não significou gravação de todos os aceites. Na síncrona, foram aceitos e persistidos 1.676 dos 2.160 eventos oferecidos. Com consumidor interrompido, a assíncrona persistiu ao fim todos os 2.161 aceites. Esses casos validam o procedimento, não correspondem às cinco repetições definitivas nem confirmam hipóteses comparativas. Catálogo, nove resumos, comparação independente do modelo e hashes de duzentos arquivos brutos foram preservados no relatório R2.

5.2 MODELO COMUM DE REGISTROS: OBJETIVO A

Em 6 de outubro de 2026, o teste de equivalência enviou um conjunto determinístico de 50 registros a cada variante, em execuções separadas. Os UUIDs, campos, instantes de ocorrência e payloads foram iguais; somente o identificador de execução, transportado fora do conteúdo canônico, diferiu. Ambas as exportações continham 50 linhas. A comparação independente confirmou igualdade dos identificadores, campos, payloads e SHA-256, desconsiderando apenas `persisted_at`, pois as inserções ocorreram em momentos distintos.

O reenvio do primeiro registro, com o mesmo conteúdo, produziu um marco de persistência `duplicate` em cada variante e não aumentou a contagem de linhas. Em seguida, o payload desse UUID foi alterado de zero para nove. A API síncrona respondeu 409. A assíncrona respondeu 202 por confirmar a publicação; o consumidor identificou `event_id_content_conflict` e confirmou o encaminhamento da mensagem conflitante à DLQ. O conteúdo original no banco permaneceu inalterado.

Esse resultado evidencia equivalência funcional para o conjunto examinado e verifica os caminhos de idempotência e conflito. Não demonstra equivalência para todos

os possíveis payloads ou condições de concorrência e não constitui comparação de desempenho. As execuções válidas foram identificadas pelos prefixos `e71d7147` e `6cdf03fd`; os UUIDs completos, as exportações e seus checksums constam do catálogo técnico de evidências.

5.3 VARIANTE SÍNCRONA: OBJETIVO B

A implementação síncrona recebe o evento pela API REST e conclui a transação antes de devolver 201. Nos quatro níveis constantes e na rajada, todos os envios medidos obtiveram a resposta esperada e foram persistidos ao fim. Com PostgreSQL parado, houve 2.452 respostas 503 nas cinco tentativas, correspondentes a uma média de 22,70% de erro por execução. Os 8.351 aceites foram persistidos; uma requisição adicional com 503 também deixou registro no banco. Esse último caso mostra por que erro de transporte/aplicação e ausência de persistência não são sinônimos.

Não houve UUID duplicado no banco ou divergência de conteúdo/hash nos registros medidos. A unicidade observada é compatível com o repositório idempotente, mas a campanha utiliza UUIDs novos: as provas de reenvio idempotente e conflito pertencem à validação funcional, não são inferidas de uma carga que não os injeta deliberadamente. O fluxo síncrono mais curto permite confirmar o resultado ao cliente no sucesso; em falha, o cliente precisa de conciliação antes de repetir uma operação cujo resultado seja incerto.

5.4 VARIANTE ASSÍNCRONA: OBJETIVO C

O aceite e a persistência são etapas distintas. Todos os envios medidos da assíncrona obtiveram 202, mas isso não implicou persistência de todos os aceites. Com PostgreSQL parado, 14 dos 10.804 aceites terminaram na DLQ, distribuídos como 3, 3, 3, 3 e 2 entre os blocos. Os outros 10.790 foram persistidos. A política de tentativas limitadas explica o destino terminal previsto, não uma garantia de gravação futura. A retomada do banco não reprocessa automaticamente a DLQ.

Com o consumidor parado, os 10.801 envios medidos foram aceitos e persistidos ao fim, sem DLQ. O cruzamento temporal verifica, em cada repetição, aceites enquanto o consumidor estava parado e seus commits depois da retomada. Isso sustenta retenção temporária nesse cenário, não tolerância à perda do broker ou da máquina. O p95 de conclusão cresce para dezenas de segundos, enquanto o HTTP permanece na escala de milissegundos; portanto, o benefício de aceite não deve ser apresentado como conclusão mais rápida.

5.5 EXECUÇÃO DOS CENÁRIOS: OBJETIVO D

A campanha contém treze condições com cinco tentativas cada: quatro taxas constantes, 12, 24, 36 e 48 eventos/s, nas duas variantes; rajada 48–96–48 nas duas; inter-

rupção do PostgreSQL nas duas; e interrupção do consumidor somente na assíncrona. São cinco blocos, trinta pares com dataset comum e cinco execuções adicionais de consumidor parado. A semente de ordem foi 20261026. Cada coleta manteve 60 s de aquecimento, intervalo de 12 s, 180 s medidos e 90 s de observação posterior. As falhas solicitaram 40 s de parada a aproximadamente 60 s da medição, com instantes efetivos registrados.

O protocolo original foi publicado antes da campanha, no commit `f5d77b5`. Na 37ª tentativa, UUID `8733a7d2-89b5-4a5e-be22-0e4ed6e6b29b`, o analisador encontrou um registro correto no banco após HTTP 503, sem marco pós-commit. A coleta síncrona com banco parado foi interrompida como inválida no critério original. A revisão formal, publicada em `effa20b`, preservou os brutos e as 36 anteriores. A A1 foi autorizada, testada, auditada e publicada em `cb3e9b3`, antes de retomar somente os 28 UUIDs nunca iniciados. A decisão é pós-hoc nas primeiras 37 observações e anterior às últimas 28. Não houve reconstrução de imagens, mudança de taxa, substituição de UUID ou terceira tentativa silenciosa.

A coleta parcial mantém `instrument_valid=false`. Seus indicadores HTTP, destinos e recursos são utilizados após auditoria; não há preenchimento do instante de commit. A comparação primária de latência de conclusão exclui o par do bloco 3 com banco parado, restando quatro execuções em cada variante nesse indicador. Demais comparações identificadas mantêm cinco. A execução parcial teve 1.666 tempos observados para 1.667 registros persistidos; a ausência ocorreu na fronteira da falha e não é assumida aleatória.

O controlador encerrou às 17h01 de Brasília e restaurou o projeto interativo. A auditoria final conferiu a união exata dos 65 UUIDs, sem interseção entre completas e parcial, cinco tentativas por condição, datasets por par, manifestos, fingerprints e checksums antes/depois da leitura. Os pareceres finais reproduzem os registrados durante a campanha, sem alteração de resumos originais. Pilotos, calibrações, a R1 rejeitada e demonstrações não integram esse catálogo.

Foram verificados 1.495 arquivos brutos catalogados, antes e depois da leitura. Os manifestos confirmam 37 coletas sob o commit original e 28 sob a publicação A1, com os mesmos fingerprints medidos. Entre confirmação da parada e conclusão do comando de retomada, o intervalo observado foi de 40,65 a 41,11 s no PostgreSQL e de 45,02 a 46,52 s no consumidor; a solicitação de 40 s não representa exatamente a indisponibilidade efetiva. Reinício e prontidão posteriores também integram os tempos até os marcos observados.

O conjunto público fica em `<implementacao-java/experimento/campanhas/2026-10-27-reduzida65-r2/resultados-a1/>`. Contém catálogo, resumos originais, pareceres derivados, manifestos sanitizados, índices de hashes, valores por execução, distribuições descritivas e diferenças pareadas. Os arquivos `per-run.csv`, `group-summary.json` e `paired.json` permitem recuperar cada repetição, média, mediana, desvio-padrão amostral, mínimo e máximo; os milhares de eventos de uma execução nunca são usados como tamanho amostral independente. Os brutos integrais permanecem preservados localmente, sem publicar indiscriminadamente logs ou caminhos pessoais.

5.6 MÉTRICAS COMPARATIVAS: OBJETIVO E

5.6.1 Resposta HTTP e conclusão da persistência

TABELA 9 – Latência: média e desvio-padrão dos p95 de cada execução, em ms

Condição	Var.	n_H	HTTP válido	n_C	Envio → commit
Constante 12/s	S	5	7,00 ± 1,44	5	6,20 ± 1,42
Constante 12/s	A	5	4,27 ± 0,54	5	8,17 ± 1,63
Constante 24/s	S	5	5,52 ± 1,05	5	4,94 ± 0,88
Constante 24/s	A	5	3,40 ± 0,49	5	6,55 ± 0,77
Constante 36/s	S	5	4,89 ± 0,81	5	4,40 ± 0,69
Constante 36/s	A	5	2,92 ± 0,37	5	6,68 ± 1,70
Constante 48/s	S	5	4,34 ± 0,30	5	3,92 ± 0,25
Constante 48/s	A	5	2,60 ± 0,26	5	5,83 ± 1,19
Rajada	S	5	4,51 ± 0,64	5	4,06 ± 0,57
Rajada	A	5	3,03 ± 0,91	5	6,66 ± 2,37
Banco parado	S	5	19,46 ± 12,02	4	21,91 ± 11,43
Banco parado	A	5	4,13 ± 0,41	4	36406,01 ± 662,82
Consumidor parado	A	5	4,58 ± 0,82	5	49905,09 ± 4611,67

FONTE: Elaborado pelo autor a partir da campanha R2/A1 de 27/10/2026.

S: síncrona; A: assíncrona. n_H e n_C contam execuções, não eventos. HTTP inclui somente 201/202 com UUID correto. A latência de conclusão usa apenas marcos conciliados; o par do bloco 3 com banco parado é excluído de ambas as variantes nesse indicador. Média de p95 não é o p95 de eventos agrupados.

TABELA 10 – Diferenças pareadas do p95 HTTP: assíncrona menos síncrona, em ms

Condição	Bloco 1	Bloco 2	Bloco 3	Bloco 4	Bloco 5
Constante 12/s	-2,26	-2,80	-1,77	-4,54	-2,28
Constante 24/s	-1,59	-1,76	-2,44	-1,60	-3,20
Constante 36/s	-1,53	-1,67	-1,60	-1,59	-3,47
Constante 48/s	-1,90	-1,60	-1,98	-1,44	-1,75
Rajada	-0,24	-1,60	-1,72	-1,25	-2,56
Banco parado	-17,01	-32,12	-2,90	-20,08	-4,54

FONTE: Elaborado pelo autor a partir da campanha R2/A1 de 27/10/2026.

Cada coluna é um par com o mesmo dataset, não uma amostra de evento. Diferenças negativas indicam menor p95 HTTP da assíncrona nestes cinco pares. Não se aplica teste de significância; todas as diferenças, dispersões e diferenças de conclusão/recursos constam de `paired.json`. Não existe par síncrono para consumidor parado.

Nas cinco condições sem falha, os cinco pares de cada condição apresentaram menor p95 HTTP e maior p95 envio–commit na assíncrona. Por exemplo, a 48/s, as médias dos p95 HTTP foram 2,60 ms na assíncrona e 4,34 ms na síncrona; para conclusão, 5,83 ms e 3,92 ms. A menor latência de aceite é coerente com confirmar apenas a publicação; não significa que o processamento assíncrono terminou antes. Com banco parado, os quatro pares completos mantêm a comparação de conclusão, mas não satisfazem o número de cinco pares previsto para uma conclusão direcional completa nessa condição.

O p95 não descreve sozinho toda a cauda. No banco parado síncrono, a média das médias de HTTP válido foi aproximadamente 99,17 ms, apesar da média dos p95 de

19,46 ms: menos de 5% das respostas longas podem elevar a média sem elevar o p95 na mesma proporção. Os p99 e máximos por execução estão preservados nos resumos. Isso não é contradição entre indicadores nem razão para escolher apenas o mais favorável.

5.6.2 Vazão, erros e destinos

TABELA 11 – Vazão concluída na janela e erros HTTP por execução

Condição	Var.	Vazão média (eventos/s)	Erro HTTP (%)
Constante 12/s	S	12,00000 ± 0,00000	0,00 ± 0,00
Constante 12/s	A	12,00000 ± 0,00000	0,00 ± 0,00
Constante 24/s	S	24,00000 ± 0,00000	0,00 ± 0,00
Constante 24/s	A	24,00000 ± 0,00000	0,00 ± 0,00
Constante 36/s	S	36,00000 ± 0,00000	0,00 ± 0,00
Constante 36/s	A	36,00000 ± 0,00000	0,00 ± 0,00
Constante 48/s	S	48,00000 ± 0,00000	0,00 ± 0,00
Constante 48/s	A	48,00000 ± 0,00000	0,00 ± 0,00
Rajada	S	63,99444 ± 0,00000	0,00 ± 0,00
Rajada	A	63,99444 ± 0,00000	0,00 ± 0,00
Banco parado	S	[9,27556; 9,27667]	22,70 ± 0,25
Banco parado	A	11,98444 ± 0,00248	0,00 ± 0,00
Consumidor parado	A	12,00000 ± 0,00000	0,00 ± 0,00

FONTE: Elaborado pelo autor a partir da campanha R2/A1 de 27/10/2026.

$n = 5$ em todas as linhas. Valores pontuais mostram média ± desvio-padrão entre execuções; a única linha intervalar mostra médias dos limites inferior/superior, não intervalo de confiança. Os limites e a dispersão individuais estão no catálogo. Drenagem não amplia os 180 s. Ausência de diferença de vazão nesta carga não estabelece equivalência de capacidade.

TABELA 12 – Destinos ao fim da observação: totais das cinco tentativas por condição

Condição	Var.	Enviados	Aceitos	Banco	DLQ	Não banco
Constante 12/s	S	10804	10804	10804	0	0
Constante 12/s	A	10805	10805	10805	0	0
Constante 24/s	S	21604	21604	21604	0	0
Constante 24/s	A	21605	21605	21605	0	0
Constante 36/s	S	32405	32405	32405	0	0
Constante 36/s	A	32405	32405	32405	0	0
Constante 48/s	S	43204	43204	43204	0	0
Constante 48/s	A	43205	43205	43205	0	0
Rajada	S	57595	57595	57595	0	0
Rajada	A	57595	57595	57595	0	0
Banco parado	S	10803	8351	8352	0	2451
Banco parado	A	10804	10804	10790	14	14
Consumidor parado	A	10801	10801	10801	0	0

FONTE: Elaborado pelo autor a partir da campanha R2/A1 de 27/10/2026.

Não banco é enviado menos persistido, e não sinônimo de perda. No banco parado síncrono, 2452 respostas foram 503; uma delas corresponde a registro persistido, sem marco temporal. Restam 2451 enviados não aceitos e não persistidos. Na assíncrona, os 14 aceitos não persistidos terminaram em DLQ. Em todas as linhas: pendência retida final, aceites sem destino conciliado, UUIDs duplicados no banco e divergências de conteúdo/hash iguais a zero.

Nas cargas constantes, ambas concluíram a taxa programada dentro da janela, sem erro HTTP medido. Na rajada, ambas concluíram em média 63,99444 eventos/s nos 180 s, sem provar limite máximo ou equivalência entre arquiteturas. Houve onze iterações não iniciadas no aquecimento da primeira execução assíncrona de rajada, conforme os pontos brutos `dropped_iterations` com cenário `warmup`; não foram drops da janela medida. A ocorrência é preservada no resumo de todas as fases e não foi motivo para repetir a coleta. Contagem oferecida efetiva, fase e janela devem ser distinguidas de uma demanda teórica arredondada.

Na coleta parcial, a vazão possui os seguintes limites conservadores, em eventos/s:

$$[1666/180; 1667/180] = [9{,}25556; 9{,}26111].$$

A média dos limites nas cinco tentativas síncronas de banco parado é $[9{,}27556; 9{,}27667]$. A assíncrona concluiu 11,98444 eventos/s em média, mas reteve 14 resultados terminais na DLQ. Não houve aceites sem destino conciliado nem pendência retida ao fim; não se conclui que inexistem modos de perda fora da janela, nem se confunde DLQ com desaparecimento ou persistência.

5.6.3 CPU e memória

TABELA 13 – Recursos do caminho de processamento: média e desvio-padrão das médias por execução

Condição	Var.	CPU (%)	Memória (MiB)
Constante 12/s	S	6,79 ± 0,56	231,0 ± 4,1
Constante 12/s	A	15,36 ± 1,29	1020,5 ± 11,0
Constante 24/s	S	10,87 ± 1,16	237,0 ± 1,8
Constante 24/s	A	25,53 ± 5,08	1018,8 ± 22,2
Constante 36/s	S	12,29 ± 0,50	240,7 ± 3,8
Constante 36/s	A	27,90 ± 4,56	1035,2 ± 21,2
Constante 48/s	S	14,45 ± 0,52	244,8 ± 3,4
Constante 48/s	A	30,94 ± 1,57	1044,9 ± 15,9
Rajada	S	18,57 ± 2,19	244,5 ± 1,4
Rajada	A	39,86 ± 6,48	1048,3 ± 25,3
Banco parado	S	7,13 ± 0,69	230,0 ± 3,3
Banco parado	A	15,81 ± 3,14	1016,9 ± 21,9
Consumidor parado	A	22,23 ± 5,35	966,3 ± 22,8

FONTE: Elaborado pelo autor a partir da campanha R2/A1 de 27/10/2026.

$n = 5$. Somam-se amostras simultâneas de API e PostgreSQL na síncrona; API, consumidor, PostgreSQL e Kafka na assíncrona. Gerador e observador são separados. 100% de CPU corresponde a uma CPU lógica, não ao host inteiro. São médias aritméticas das amostras; componentes parados e o custo da telemetria limitam a interpretação, sem provar custo financeiro ou eficiência por trabalho realizado.

A assíncrona utilizou mais CPU e memória do caminho completo em todos os cinco pares de cada condição comparável. A 12/s constante, foram 15,36% contra 6,79% de CPU e aproximadamente 1.020,5 contra 231,0 MiB. O broker e o consumidor adicionais integram a topologia comparada; a diferença não pode ser atribuída somente a um

protocolo abstrato. A soma não inclui gerador ou observador, cujas margens são verificadas separadamente nos resumos e no catálogo. Médias menores durante parada não demonstram maior eficiência: componentes podem estar deixando de processar trabalho.

Entre os definitivos, os maiores p95 de CPU do gerador e observador foram, respectivamente, 18,665% e 9,56% das quotas; seus máximos de memória foram 23,965% e 45,391% dos limites. Essas margens não são consumo do caminho de negócio nem prova de ausência de qualquer interferência. As amostras de recursos são somadas por lote temporal comum, sem preencher componentes ausentes com zero.

5.6.4 Recuperação e persistência após falha

TABELA 14 – Recuperação observada após o comando de retomada, em segundos

Condição	Var.	Primeiro commit	Estável: $n/5$	Confirmação	Backlog completo
Banco parado	S	4,92 ± 0,47	5/5	38,47 ± 0,57	–
Banco parado	A	4,44 ± 1,08	1/5	38,71	–
Consumidor parado	A	11,82 ± 3,59	5/5	41,63 ± 7,54	15,36 ± 5,00

FONTE: Elaborado pelo autor a partir da campanha R2/A1 de 27/10/2026.

Primeiro commit: primeiro marco observado, não prova de primeiro commit real. Confirmação exige três janelas completas de 10 s sob entrada ativa. Média/desvio são condicionados aos casos observados: banco síncrono $n = 5$, banco assíncrono $n = 1$, consumidor $n = 5$. Não se preenchem tempos nos quatro casos assíncronos sem sequência qualificante. Backlog síncrono não se aplica (zero aceites pendentes na retomada); no banco assíncrono, não houve persistência integral dos aceites anteriores por haver DLQ; consumidor teve backlog integralmente persistido em cinco casos.

Os cinco casos síncronos e os cinco casos de consumidor parado apresentaram a sequência sustentada exigida, mas somente um dos cinco casos assíncronos de banco parado a apresentou. Nos quatro restantes, houve retorno de commits e processamento posterior, porém nenhuma sequência qualificante de trinta segundos foi observada dentro da entrada ativa. Não se substitui esse resultado pelo instante do primeiro commit, pelo lag zero ou por observação durante drenagem. A ausência do critério na janela não demonstra falha permanente; tampouco autoriza declarar recuperação sustentada.

Há uma limitação de construto importante nesse critério congelado: a contagem de aceites sem commit inclui também destinos terminais em DLQ, sem subtraí-los como trabalho encerrado. A 12/s, o limite é dois. Assim, nos quatro casos com três mensagens em DLQ, esses três aceites sem commit já impedem satisfazer o limite, mesmo depois do processamento dos demais eventos. No quinto caso, duas mensagens em DLQ não impedem esse requisito e a sequência completa foi observada. Mantém-se o veredito original, mas não se interpreta o indicador composto como prova de que o serviço continuou indisponível. Uma definição alternativa que separe pendência ativa de destino terminal seria outro construto, não uma correção retroativa para aprovar os quatro casos.

No consumidor parado, a recuperação integral do backlog anterior à retomada ocorreu em cinco casos, com média de 15,36 s e extremos de 12,55 a 24,16 s. O p95 de conclusão por execução variou de 46,95 a 57,80 s, refletindo a parada e o tempo posterior

de reinício/processamento. No banco assíncrono, a DLQ impede declarar todos os aceites anteriores persistidos. Na síncrona não havia aceites pendentes na retomada; o cliente já havia recebido erros ou confirmações, o que não contabiliza requisições ainda em voo como aceites.

5.6.5 Sensibilidade e integridade temporal

A análise estrita original, publicada em `strict-sensitivity.json`, exclui o par do bloco 3 com banco parado de todos os indicadores pareados. A análise principal A1 conserva seus HTTP, destinos e recursos identificados, mas não sua latência de conclusão. A comparação dos quatro pares estritos não repõe a observação ausente nem elimina a limitação pós-hoc. Para condições sem essa coleta, os resultados estritos e principais são os mesmos.

Nos quatro pares estritos de banco parado, as médias dos p95 HTTP foram 4,14 ms na assíncrona e 22,58 ms na síncrona; CPU média de 16,33% e 7,34%, e memória de 1.007,6 e 230,2 MiB. A direção dessas diferenças permanece nos quatro pares, sem restaurar o quinto nem transformar essa sensibilidade em confirmação pelo critério de cinco diferenças. A comparação primária de conclusão já utiliza esses mesmos quatro pares e, portanto, não muda.

O arquivo `partial-latency-sensitivity.json` apresenta separadamente a distribuição dos tempos observados da coleta parcial e a cobertura de 1.666/1.667. Não é uma estimativa corrigida da distribuição completa: o commit ausente pode pertencer à cauda, e a sua posição não é conhecida. Os limites de vazão restringem apenas a contagem na janela. Nem a pequena diferença numérica desses limites nem a integridade do conteúdo recuperam o instante desconhecido.

5.7 VANTAGENS, DESVANTAGENS E HIPÓTESES: OBJETIVO F

As expectativas recebem destinos distintos, sem teste de significância ou alegação de equivalência. Menor uso básico de CPU/memória síncrono em carga reduzida foi corroborado descritivamente pelos cinco pares de 12/s. Menor complexidade operacional permanece apenas descritiva pela topologia; esforço de manutenção e custo financeiro são inconclusivos porque não foram medidos. Menor p95 de aceitação assíncrono foi corroborado nos cinco pares das seis condições comparáveis. Retenção durante consumidor parado foi corroborada nas cinco execuções desse cenário, com aceites durante a parada e destino final no banco.

A contrapartida de recursos e a distância entre aceite e conclusão também foram corroboradas no recorte: a assíncrona consumiu mais recursos e teve maior p95 de conclusão nas cinco condições sem falha. A interpretação de que aceite assíncrono asseguraria persistência integral após indisponibilidade do banco foi refutada pelas 14 mensagens em DLQ; não era garantia da implementação. Uma superioridade geral de resiliência perma-

nece inconclusiva: a assíncrona manteve aceites, mas o critério de recuperação sustentada falhou em quatro janelas de banco parado e o par primário de conclusão nessa condição tem apenas quatro observações completas.

Para a configuração estudada, a síncrona favorece simplicidade do caminho, confirmação imediata no sucesso e menor uso de recursos; vincula o produtor à disponibilidade do banco e exige conciliação diante de resultado incerto. A assíncrona permite aceitar durante parada do consumidor e absorver trabalho retido, com latência de conclusão maior, recursos adicionais e necessidade de acompanhar retry, DLQ e destinos. A escolha deve partir do compromisso de negócio: rapidez de aceite não substitui garantia de conclusão. Esses achados dialogam com a seleção contextual discutida por Kazanavičius e Mažeika (2023) e com a evidência empírica dependente da configuração de Cabane e Farias (2024), sem comparar seus números diretamente com este protótipo.

5.8 LIMITAÇÕES

O protótipo e o gerador compartilham um computador Windows com Docker em WSL. Há um único broker, uma partição e fator de replicação um; o aceite do broker não constitui teste de tolerância à perda da máquina. Os dados são sintéticos e possuem payload reduzido. A telemetria adiciona custo e as leituras de CPU e memória são amostras do contêiner, não consumo global do computador nem custo financeiro de manutenção.

Os intervalos entre processos dependem de relógios de parede. Intervalos negativos são detectáveis, mas a ausência deles não elimina a possibilidade de desvio. Além disso, componentes concorrentes e o estado da virtualização podem influenciar os ensaios. Essas restrições impedem generalizar o resultado local para todas as aplicações REST, todas as configurações Kafka ou ambientes de produção.

Cinco repetições constituem um orçamento reduzido, não um cálculo de poder; a emenda pós-hoc e a falta de um marco reduzem a cobertura de conclusão em um par. A campanha atravessou pausas operacionais documentadas e a A1, sem novo sorteio ou repetição, mas o estado de caches e da máquina não é idêntico ao longo do dia. Não se mediram custos financeiros, manutenção, recuperação de desastre, múltiplos brokers, perda de disco, partições de rede ou dados reais. A janela ativa finita deixa quatro recuperações sustentadas não observadas; não se prolongou retrospectivamente a janela para favorecer a hipótese. Essas limitações acompanham a resposta à pergunta de pesquisa.

6 CONCLUSÃO

Este trabalho comparou duas implementações de processamento de registros de auditoria: uma API REST que confirma o sucesso após persistir em PostgreSQL e uma API que confirma a publicação no Kafka, deixando a persistência para um consumidor. O contrato, a validação, o conteúdo determinístico e a regra de hash foram comuns. A comparação não isolou um protocolo abstrato: o broker e o consumidor adicionais fazem parte da topologia assíncrona e do uso de recursos observado.

A resposta à pergunta de pesquisa é que, no recorte experimentado, o desacoplamento temporal reduziu a latência de aceite e permitiu receber eventos durante a parada do consumidor, mas acrescentou recursos e distância entre aceite e conclusão. A síncrona utilizou menos CPU e memória e apresentou menor latência de conclusão nas condições sem falha, vinculando o produtor à disponibilidade do banco. Não se identificou uma alternativa universalmente superior. A interpretação depende de qual compromisso importa ao sistema: confirmar rapidamente o recebimento do trabalho, confirmar sua gravação ou preservar e acompanhar seu destino durante falhas.

6.1 OBJETIVOS E HIPÓTESES

O objetivo geral foi atendido dentro dos limites da campanha reduzida: produzir e analisar evidência experimental comparável de desempenho e resiliência. A campanha observou exatamente os 65 UUIDs planejados, em treze condições e cinco tentativas por condição, com 64 coletas completas no critério original e uma parcial qualificada. Isso não corresponde a 65 observações integralmente válidas para toda métrica. A análise de conclusão com banco parado possui quatro pares completos; os demais indicadores identificados conservam as observações elegíveis descritas no capítulo anterior.

O objetivo específico a foi atendido pela definição e verificação do modelo comum, com comparação independente de conteúdo e hash. Os objetivos b e c foram atendidos pelas variantes executáveis, seu repositório compartilhado e os fluxos distintos de confirmação. Idempotência, conflito e respostas de falha foram verificados funcionalmente; o resultado da campanha sem UUID duplicado não é utilizado como prova isolada de todos esses comportamentos. O objetivo d foi atendido pela calibração, aprovação do piloto R2, congelamento e execução dos cenários controlados, preservando as tentativas anteriores e a ordem dos pares.

O objetivo e foi atendido pela coleta conciliada de HTTP, envio–commit, vazão, erros, CPU, memória, destinos e recuperação, com a ressalva temporal da A1. A ausência de um marco não foi preenchida com horário de inserção, erro HTTP ou retomada. O objetivo f foi atendido pela discussão dos compromissos e dos resultados desfavoráveis, sem acrescentar repetições para obter um desfecho desejado. Os valores por execução,

dispersões e diferenças pareadas permanecem disponíveis no conjunto público; os brutos e seus índices permitem conferir a origem das medidas.

A expectativa de menor uso básico de recursos da síncrona em carga reduzida foi corroborada descritivamente pelos pares de 12 eventos/s. A expectativa de menor latência de aceitação da assíncrona também foi corroborada nas condições comparáveis, assim como a retenção durante indisponibilidade do consumidor. A contrapartida de recursos e o maior intervalo até a conclusão apareceram nos resultados. Essa confirmação é local e descritiva, não uma demonstração estatística para uma população de aplicações.

Os aspectos financeiros e de esforço de manutenção permanecem inconclusivos: não foram medidos e não podem ser deduzidos da quantidade de componentes ou das amostras de CPU. Da mesma forma, menor tempo HTTP não demonstrou conclusão mais rápida. A indisponibilidade do PostgreSQL produziu 14 eventos terminais em DLQ na assíncrona; portanto, a interpretação de que todo aceite seria posteriormente persistido foi refutada nesse cenário. O compromisso da implementação é reter um destino de falha, não reprocessá-lo automaticamente após a volta do banco.

Recuperação também exigiu uma interpretação específica. O retorno de commits não equivale à conclusão integral do backlog, e nenhum desses marcos substitui o critério de três janelas sustentadas sob entrada ativa. Esse critério foi observado nas cinco execuções síncronas e nas cinco de consumidor parado, mas em apenas uma das cinco assíncronas de banco parado. As outras quatro não foram convertidas em sucesso pela drenagem ou pelo lag final. O indicador congelado inclui aceites destinados à DLQ na contagem sem commit; por isso, os destinos terminais também impedem seu limiar, como discutido no capítulo anterior. Não se infere falha permanente ou indisponibilidade contínua do serviço. Assim, uma superioridade geral de resiliência não foi estabelecida.

6.2 LIMITAÇÕES E RECOMENDAÇÕES

As conclusões se limitam a um computador Windows com WSL e Docker, cargas sintéticas, versões fixadas, um broker, uma partição e fator de replicação um. Produtor, aplicações e instrumentos compartilharam recursos físicos. Aquecimento, quotas, ordem registrada e folga dos coletores reduzem algumas fontes de interferência, mas não eliminam virtualização, caches ou processos concorrentes. Os custos da emissão de logs e da amostragem integram o ambiente observado; as médias de contêineres não representam todo o consumo da máquina.

Cinco tentativas por condição são um orçamento explicitamente reduzido, não um tamanho demonstrado suficiente por cálculo de poder. Eventos correlacionados dentro de uma coleta não ampliam esse número. A A1 foi motivada por dado já observado nas primeiras 37 tentativas e declarada antes das 28 restantes; essa decisão pós-hoc limita a conclusão, mesmo com preservação dos vereditos originais, auditoria e análise estrita de sensibilidade. A falta do instante de um commit impede conhecer sua posição na

distribuição completa. A janela finita e os relógios entre processos acrescentam incerteza de observação.

Para um sistema semelhante ao protótipo, a síncrona é uma opção coerente quando a confirmação imediata de gravação e o menor caminho operacional são prioridades, desde que o produtor trate indisponibilidade e resultados incertos. A assíncrona é coerente quando se admite conclusão posterior e se deseja continuar aceitando trabalho durante parada do consumidor, desde que sejam acompanhados backlog, tentativas, DLQ e conciliação. Essas recomendações não são instruções de dimensionamento de produção e não substituem replicação no ambiente e no volume reais. Em ambas, distinguir aceite, persistência e destino terminal é mais importante do que escolher uma alternativa apenas pelo menor tempo de resposta.

6.3 TRABALHOS FUTUROS

Uma replicação pode ampliar o número de repetições e a duração da entrada ativa, com orçamento e critérios registrados antes de observar os novos dados. Máquinas separadas para gerador e serviços, novas cargas e payloads permitiriam investigar interferência e validade externa. Múltiplos brokers, replicação e falhas de rede ou de disco seriam necessários para examinar disponibilidade e durabilidade que este experimento não testou.

Outra linha é estudar políticas explícitas de reprocessamento da DLQ, limites de retry e observação de confirmação incerta, mantendo comparabilidade e sem substituir evidências desfavoráveis. Medidas específicas de energia, custo financeiro e esforço humano seriam necessárias para testar essas dimensões. A contribuição deste trabalho é, portanto, uma comparação rastreável com compromissos e limites observados, acompanhada de um protocolo e dados que permitem novas verificações, e não a eleição de uma arquitetura vencedora para qualquer contexto.

REFERÊNCIAS

AMIR-MOHAMMADIAN, S.; CHONG, S.; SKALKA, C. Correct audit logging: Theory and practice. In: **Principles of Security and Trust: 5th International Conference, POST 2016**. Springer, 2016. (Lecture Notes in Computer Science, v. 9635), p. 139–162. Disponível em: <https://doi.org/10.1007/978-3-662-49635-0_8>.

BLINOWSKI, G.; OJDOWSKA, A.; PRZYBYŁEK, A. Monolithic vs. microservice architecture: A performance and scalability evaluation. **IEEE Access**, v. 10, p. 20357–20374, 2022. Disponível em: <https://doi.org/10.1109/ACCESS.2022.3152803>.

CABANE, H.; FARIAS, K. On the impact of event-driven architecture on performance: An exploratory study. **Future Generation Computer Systems**, v. 153, p. 52–69, 2024. Disponível em: <https://doi.org/10.1016/j.future.2023.10.021>.

FIELDING, R. T.; TAYLOR, R. N. Principled design of the modern web architecture. **ACM Transactions on Internet Technology**, v. 2, n. 2, p. 115–150, 2002. Disponível em: <https://doi.org/10.1145/514183.514185>.

HELLAND, P. Life beyond distributed transactions: An apostate's opinion. In: **Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research**. [s.n.], 2007. p. 132–141. Disponível em: <https://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf>.

KARABEY AKSAKALLI, I. et al. Deployment and communication patterns in microservice architectures: A systematic literature review. **Journal of Systems and Software**, v. 180, p. 111014, 2021. Disponível em: <https://doi.org/10.1016/j.jss.2021.111014>.

KAZANAVIčIUS, J.; MAžEIKA, D. The evaluation of microservice communication while decomposing monoliths. **Computing and Informatics**, v. 42, n. 1, p. 1–36, 2023. Disponível em: <https://doi.org/10.31577/cai_2023_1_1>.

KITCHENHAM, B. A. et al. Preliminary guidelines for empirical research in software engineering. **IEEE Transactions on Software Engineering**, v. 28, n. 8, p. 721–734, 2002. Disponível em: <https://doi.org/10.1109/TSE.2002.1027796>.

KREPS, J.; NARKHEDE, N.; RAO, J. Kafka: A distributed messaging system for log processing. In: **Proceedings of the 6th International Workshop on Networking Meets Databases**. ACM, 2011. p. 1–7. Disponível em: <https://www.microsoft.com/en-us/research/wp-content/uploads/2017/09/Kafka.pdf>. Acesso em: 7 set. 2026.

LAIGNER, R. et al. Data management in microservices: State of the practice, challenges, and research directions. **Proceedings of the VLDB Endowment**, v. 14, n. 13, p. 3348–3361, 2021. Disponível em: <https://doi.org/10.14778/3484224.3484232>.

NIEDERMAIER, S. et al. On observability and monitoring of distributed systems – an industry interview study. In: **Service-Oriented Computing: 17th International Conference, ICSOC 2019**. Springer, 2019. (Lecture Notes in Computer Science, v. 11895), p. 36–52. Disponível em: <https://doi.org/10.1007/978-3-030-33702-5_3>.

PAUTASSO, C.; ZIMMERMANN, O.; LEYMANN, F. RESTful web services vs. “big” web services: Making the right architectural decision. In: **Proceedings of the 17th International Conference on World Wide Web**. ACM, 2008. p. 805–814. Disponível em: <https://doi.org/10.1145/1367497.1367606>.